



딥러닝 기반 딸기 숙성도 분류 및 로봇팔을 활용한 수확 자동화

지도 교수님: 이현빈 교수님

팀 원: 20202066 안다은

20201737 백민우

20222008 조민서

20222278 박동현



목차

01 시스템 개요



02 탐지 파트



03 제어 파트



04 진행 상황



05 미완 부분 정리



06 향후 개선 방향 & 결론

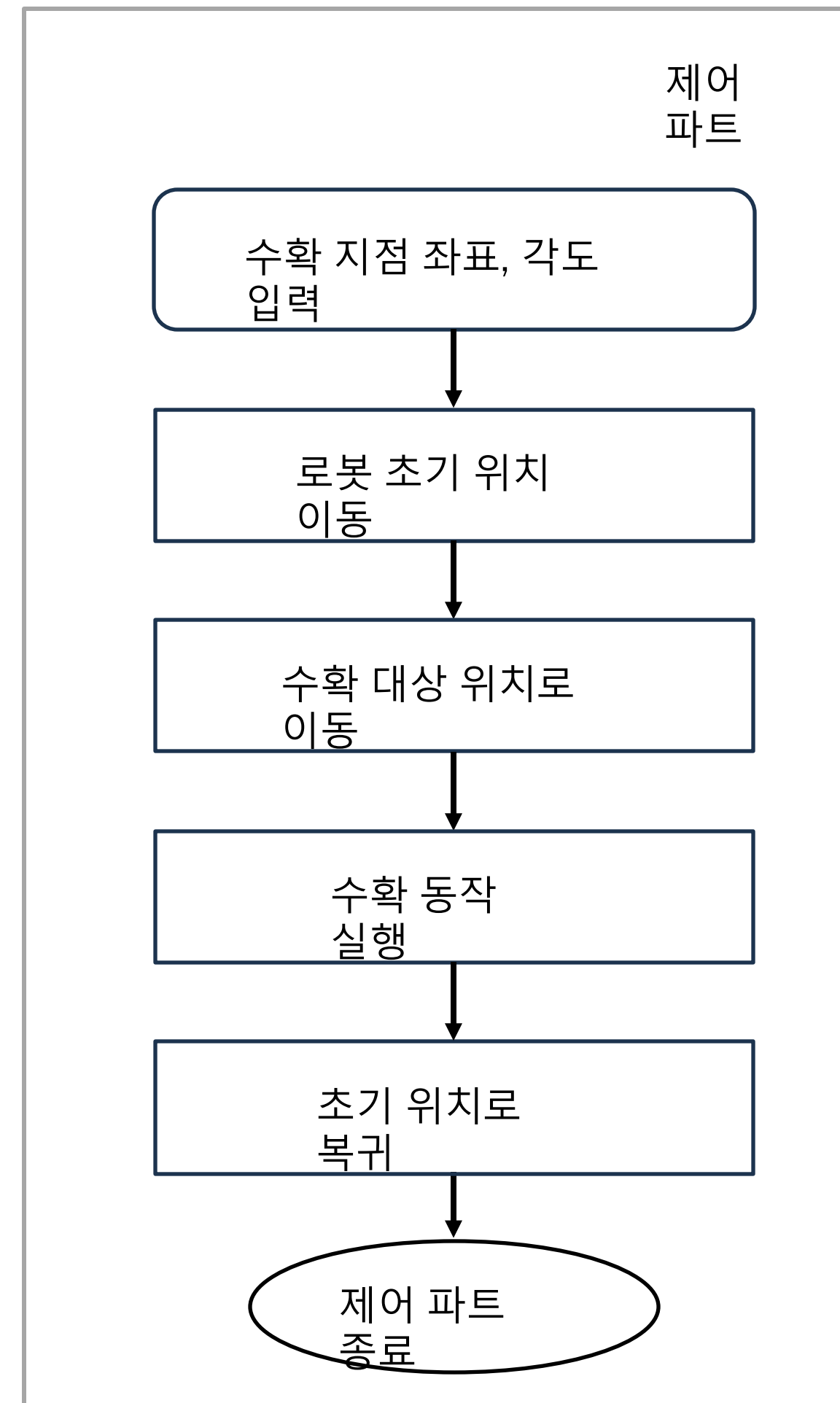
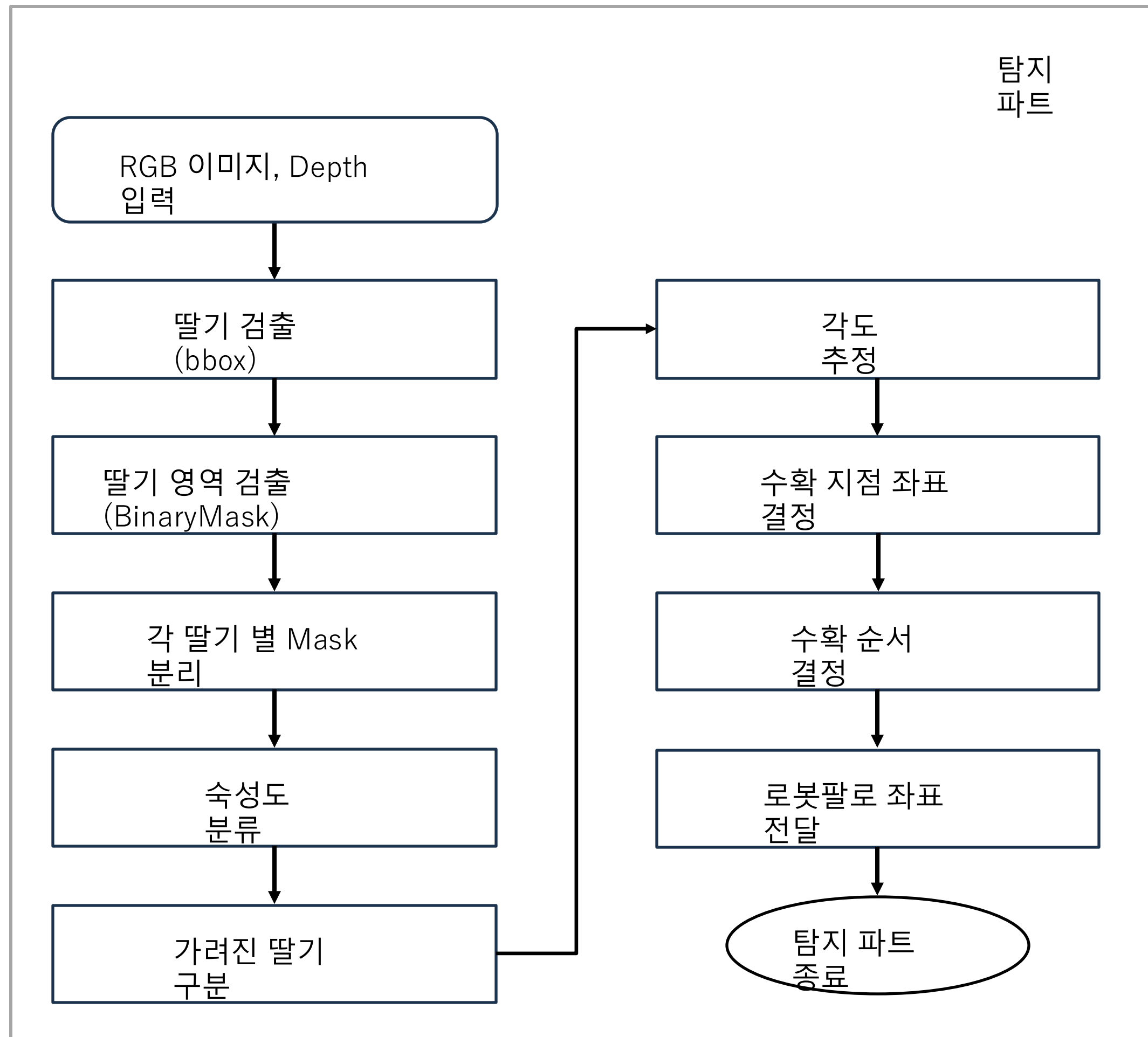


07 최종 시스템 구성도





시스템 개요





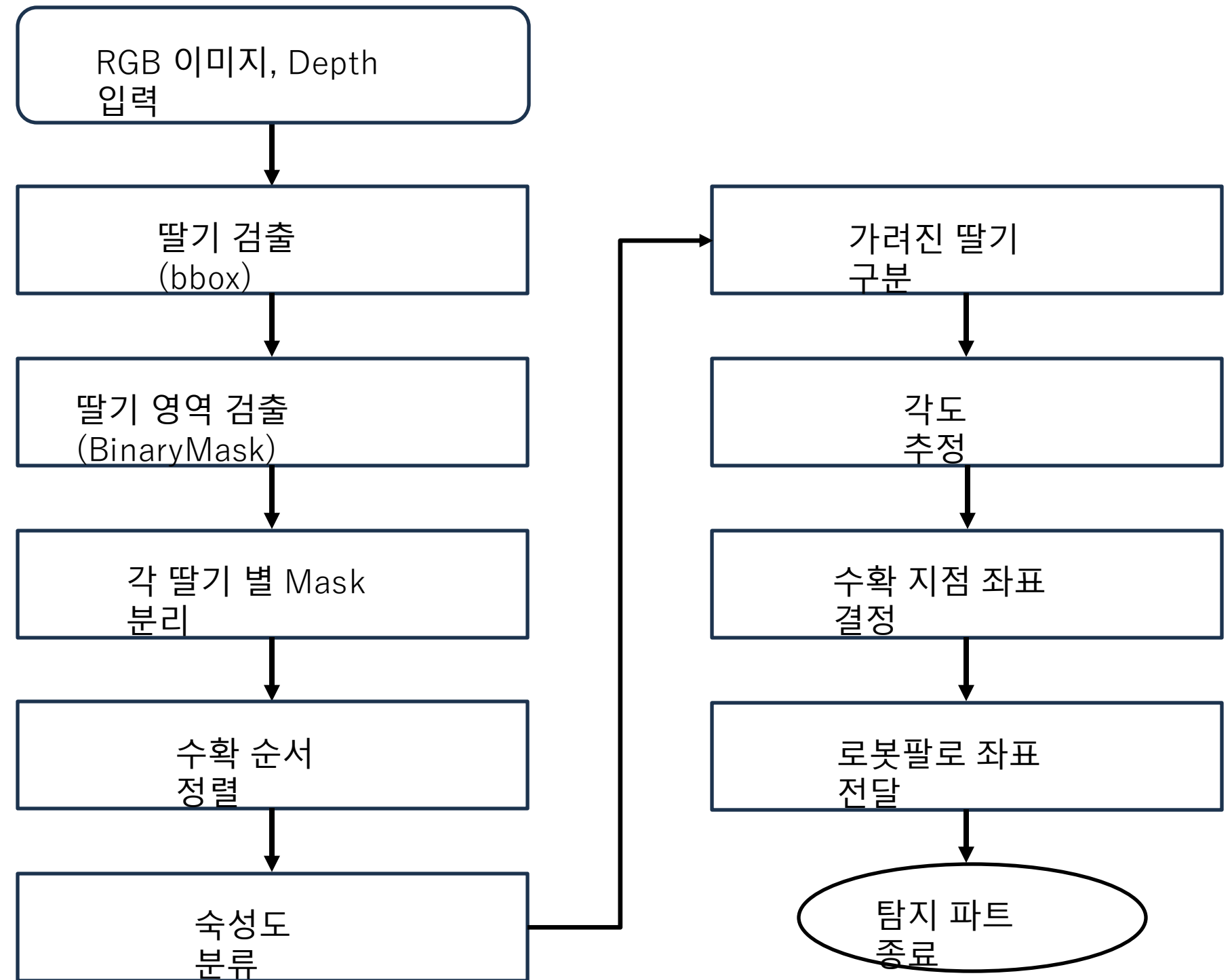
탐지 파트



탐지 파트

딸기 탐지 파트 목표

- 수확 대상 딸기의 각도, 수확 지점 좌표를 제어 파트에 전달
- Jetson orin nano에서 구동
- 로봇 팔이 고정 위치에 머무는 시간 최소화



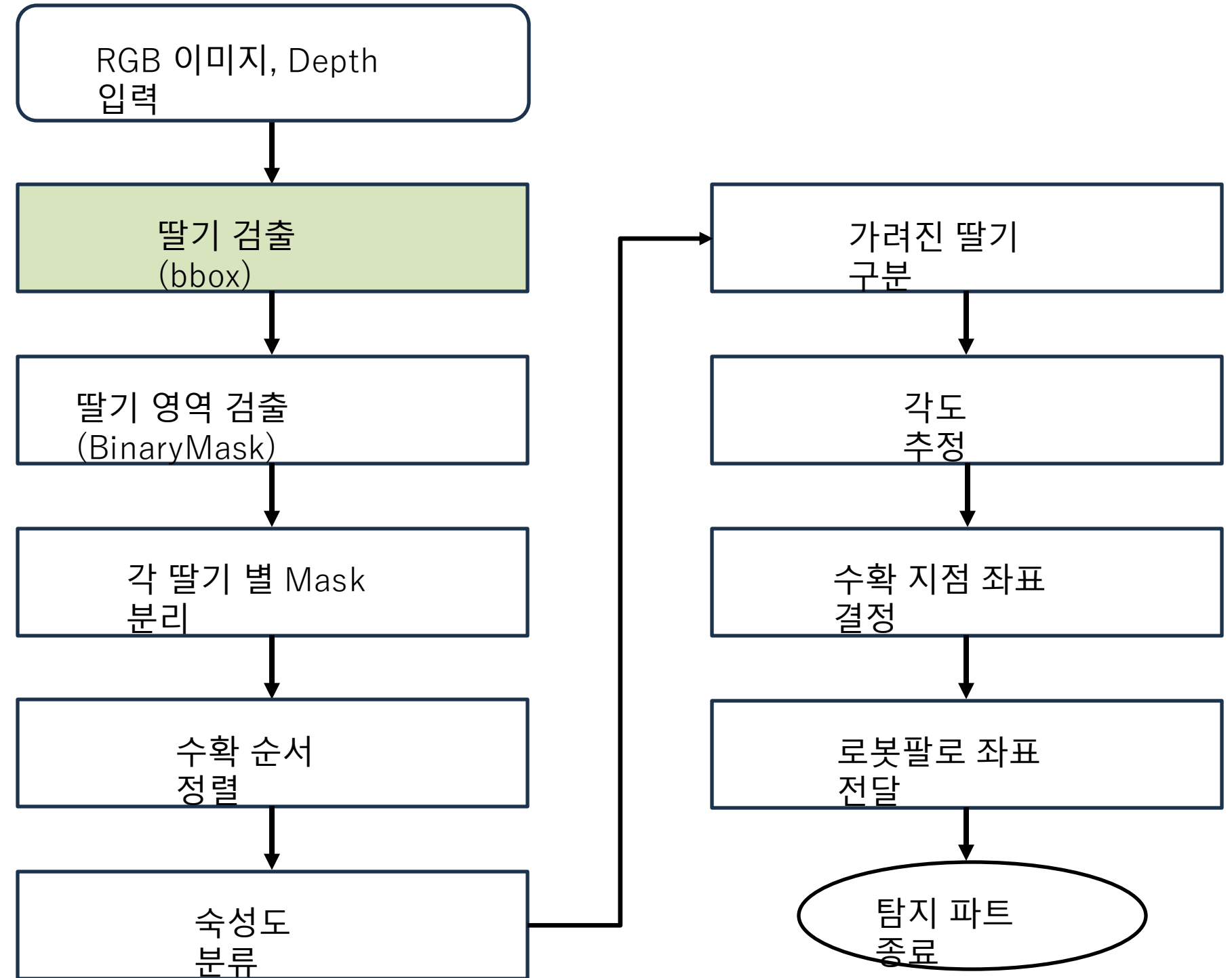


탐지 파트

딸기 Bounding Box 검출

- YOLOv5n 사용
- 학습 데이터 : 딸기 이미지 353장
- 테스트 데이터 : 딸기 이미지

40장

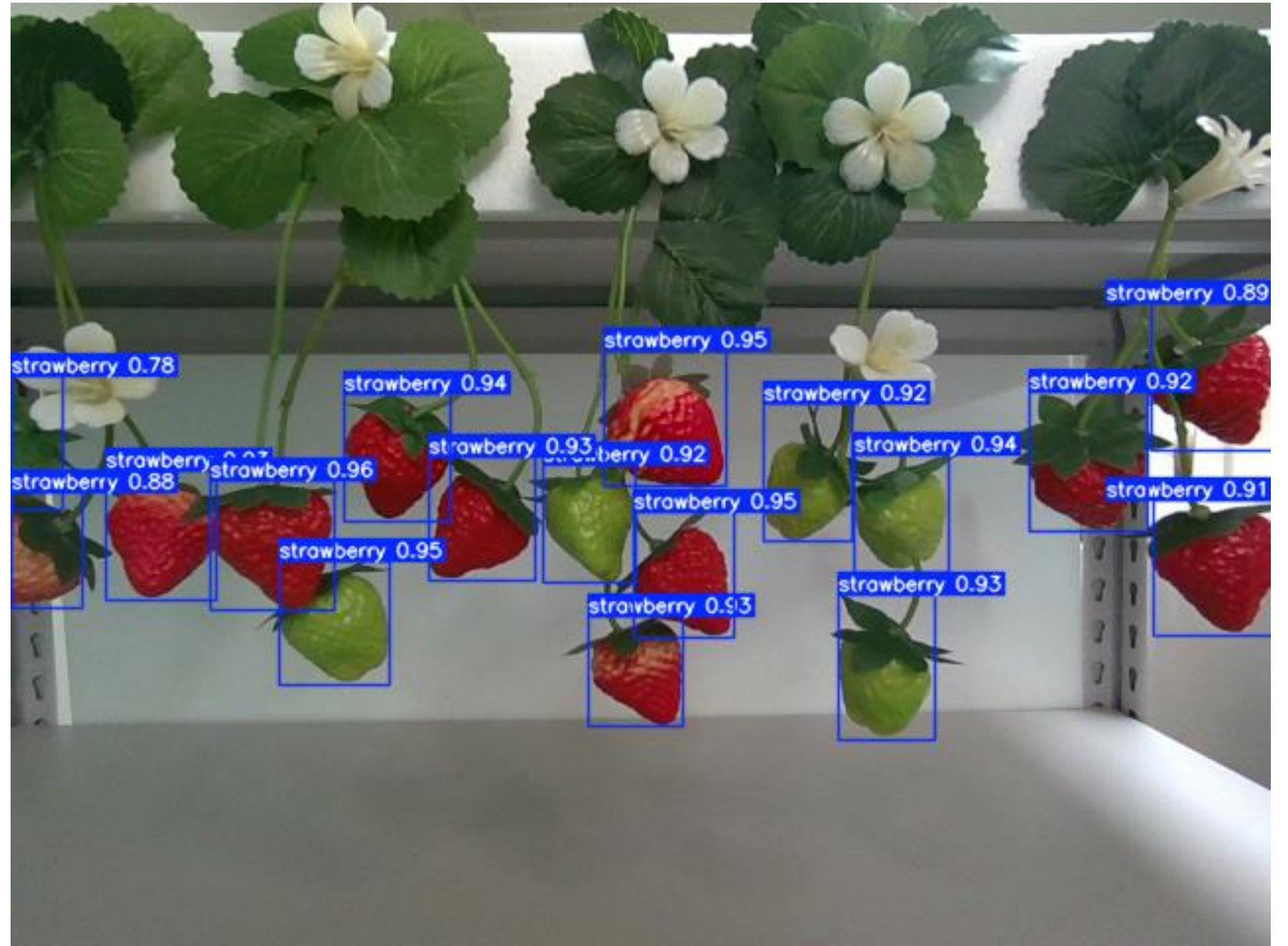




탐지 파트

테스트 결과

- Precision, 0.995
- Recall, 0.993
- mAP@0.50 = 0.995

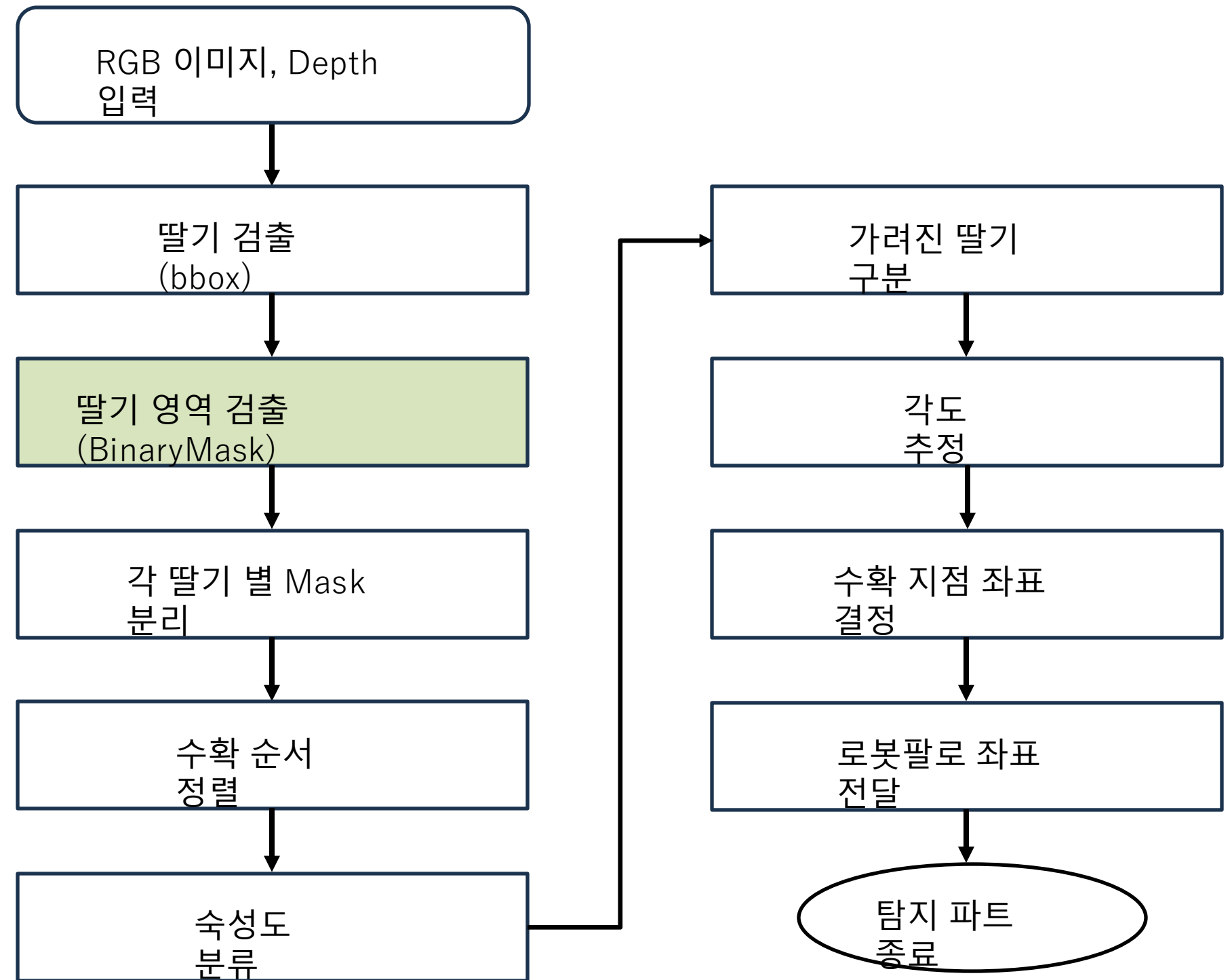




탐지 파트

딸기 영역 검출 (BinaryMask)

- mobilenet_v2 UNet 기반 사용
- 학습 데이터 : 딸기 개별 이미지 5040장
- 테스트 데이터 : 딸기 개별이미지 562장

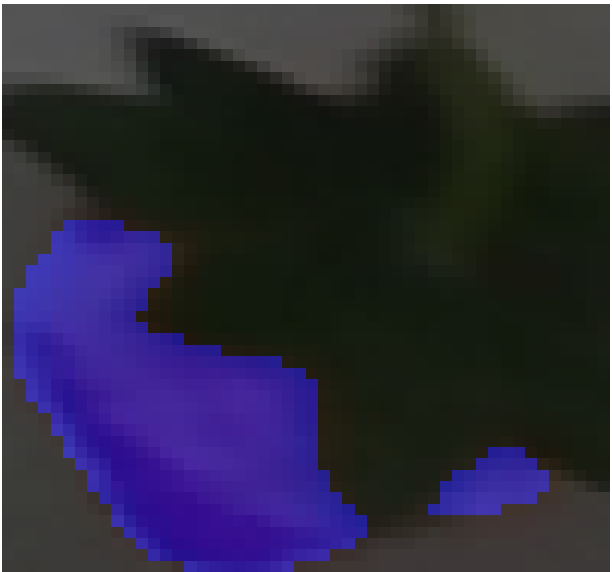
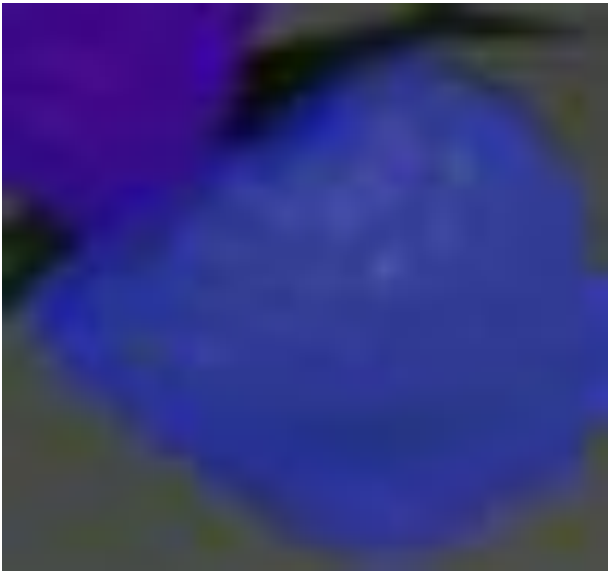
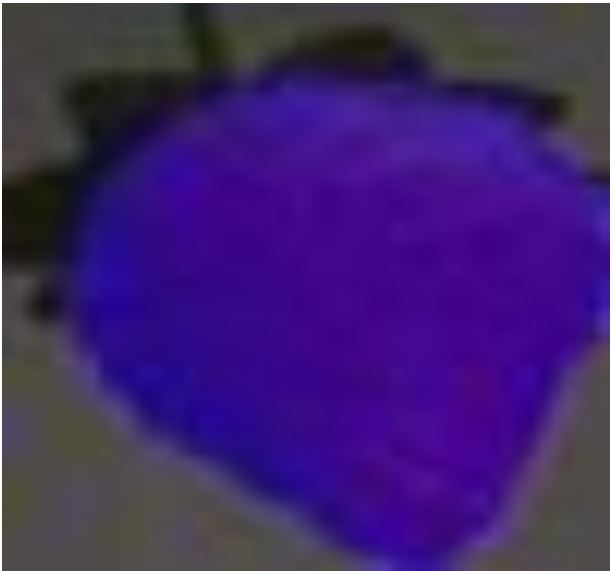




탐지 파트

테스트 결과

- Mean IoU: 0.9436

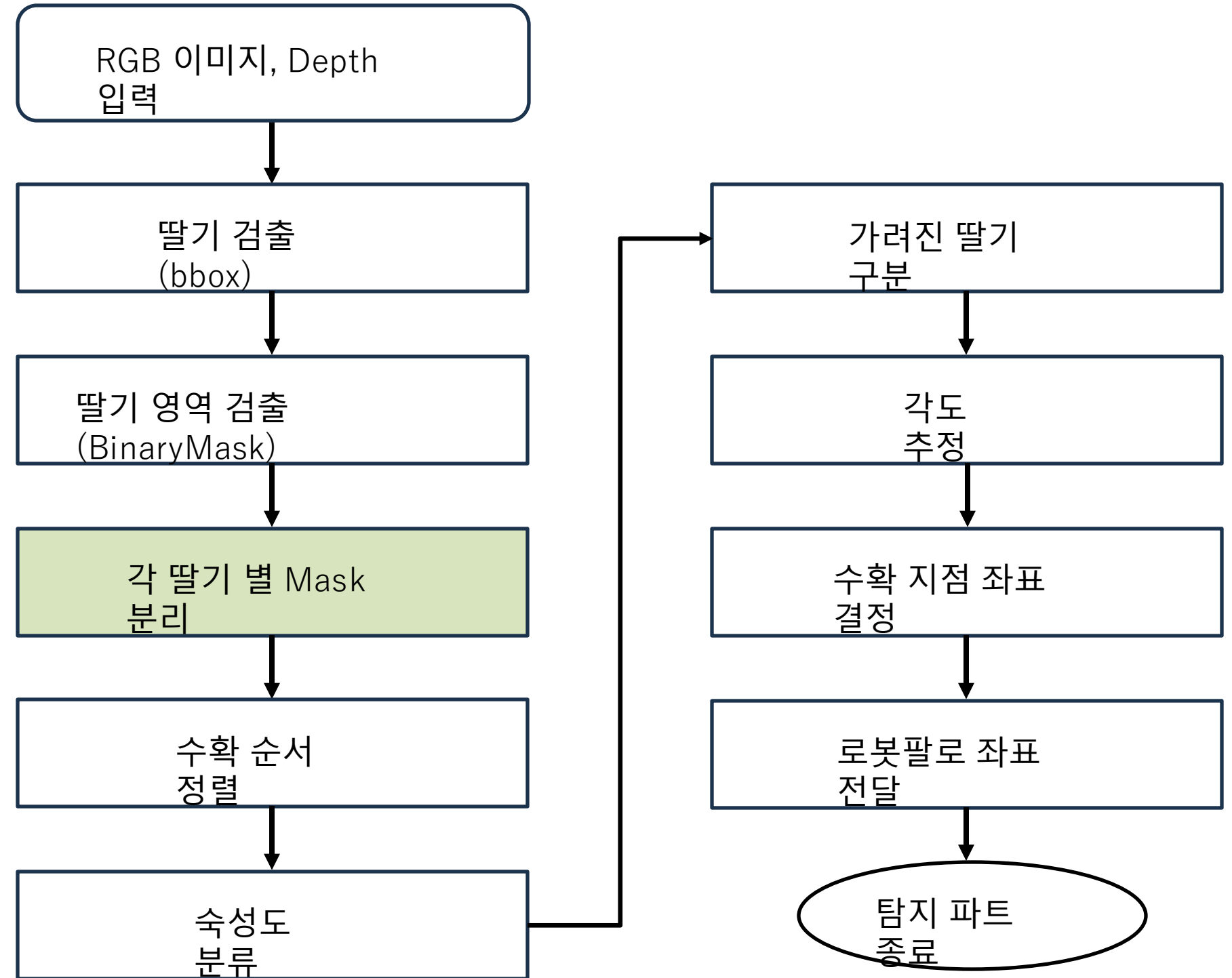




탐지 파트

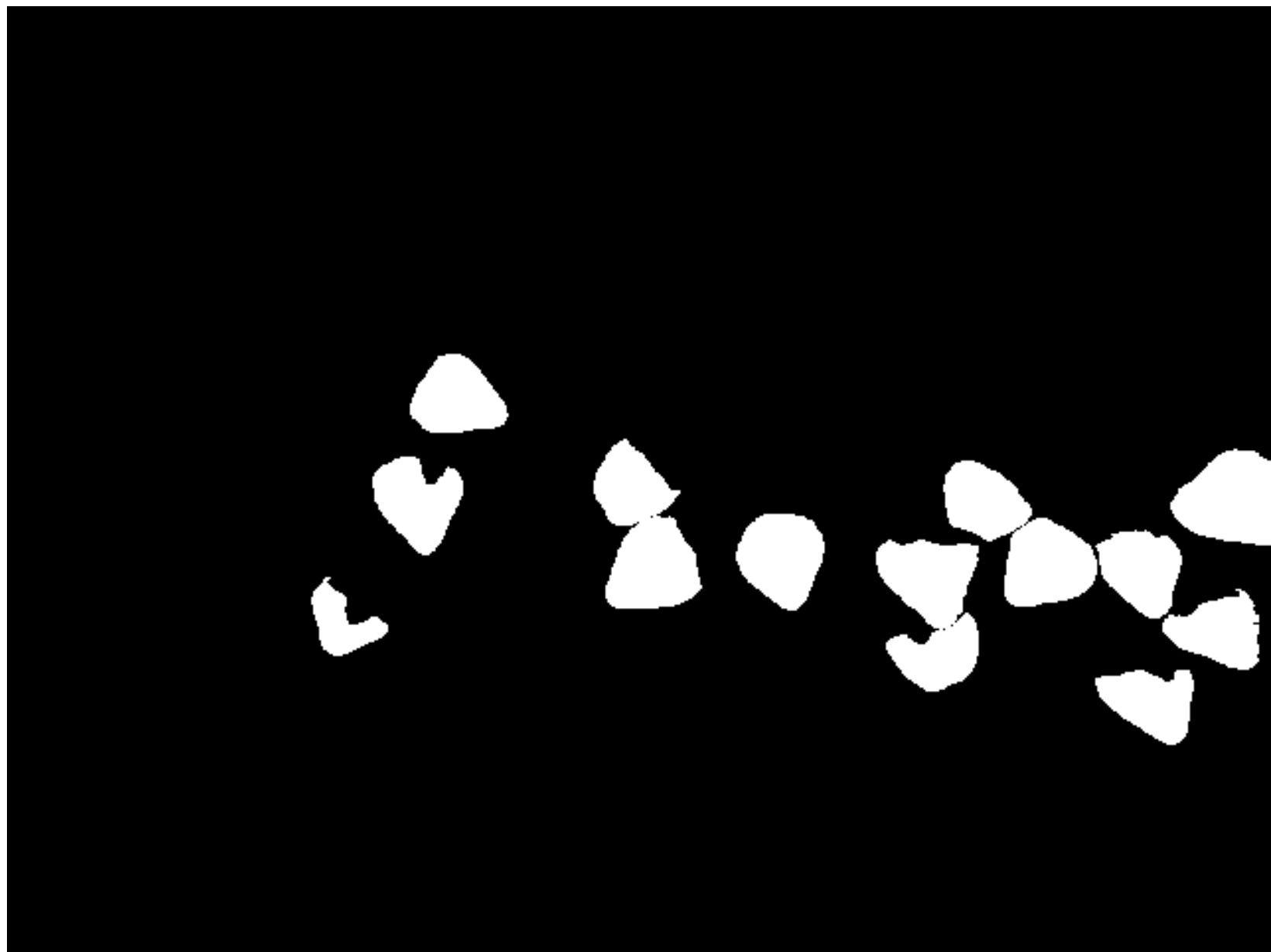
각 딸기 별 Mask 분리

1. Morphological Opening
2. Distance Transform
3. Thresholding
4. Connected Components
5. Watershed





탐지 파트

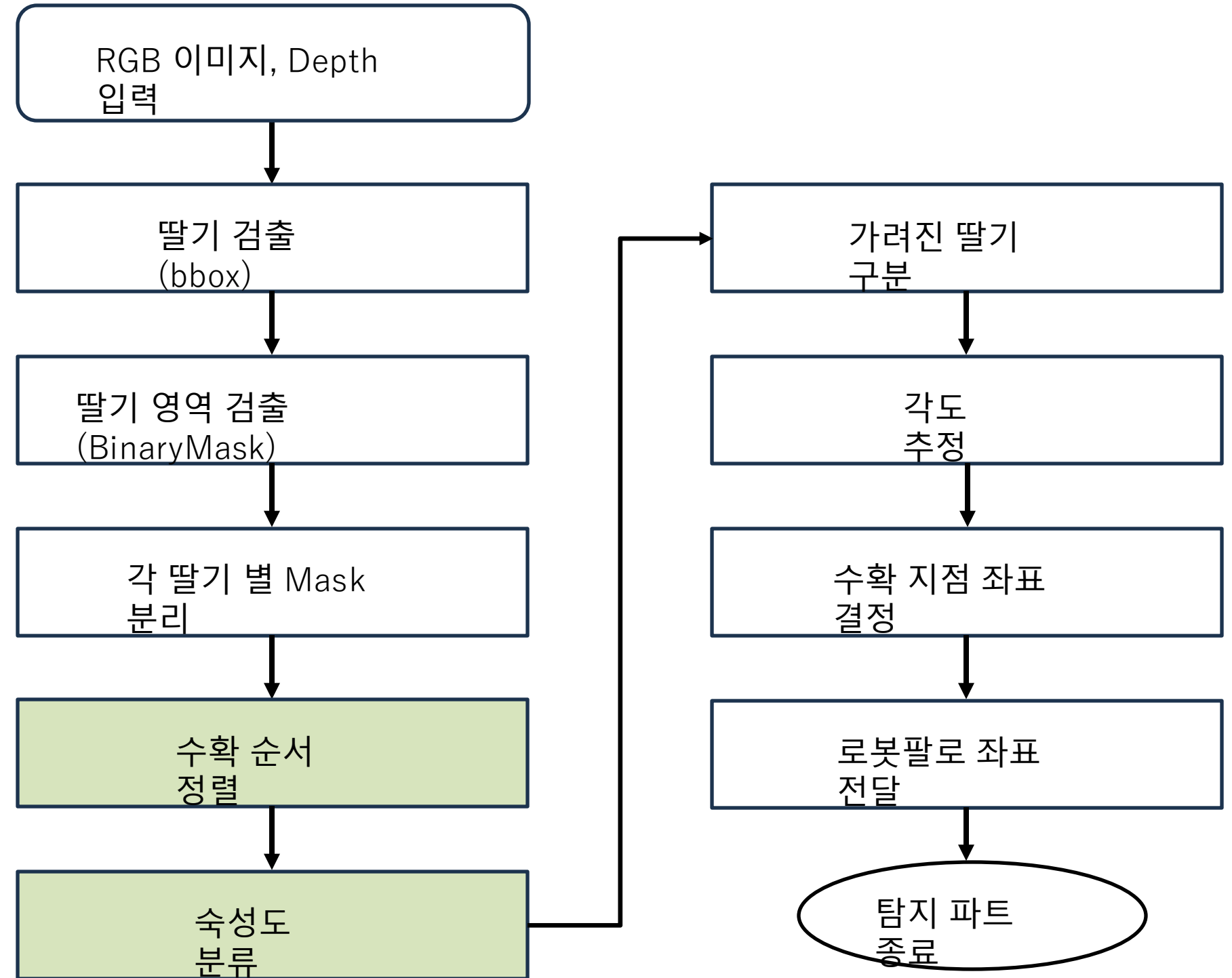




탐지 파트

숙성도 분류

- 숙성 / 미숙성 분류





탐지 파트

숙성도 분류

1. RGB -> HSV 변환
2. 유효 픽셀 필터링 (V, S)
3. H 값을 이용해 빨간색 영역 추출
4. 빨간 픽셀 비율 계산

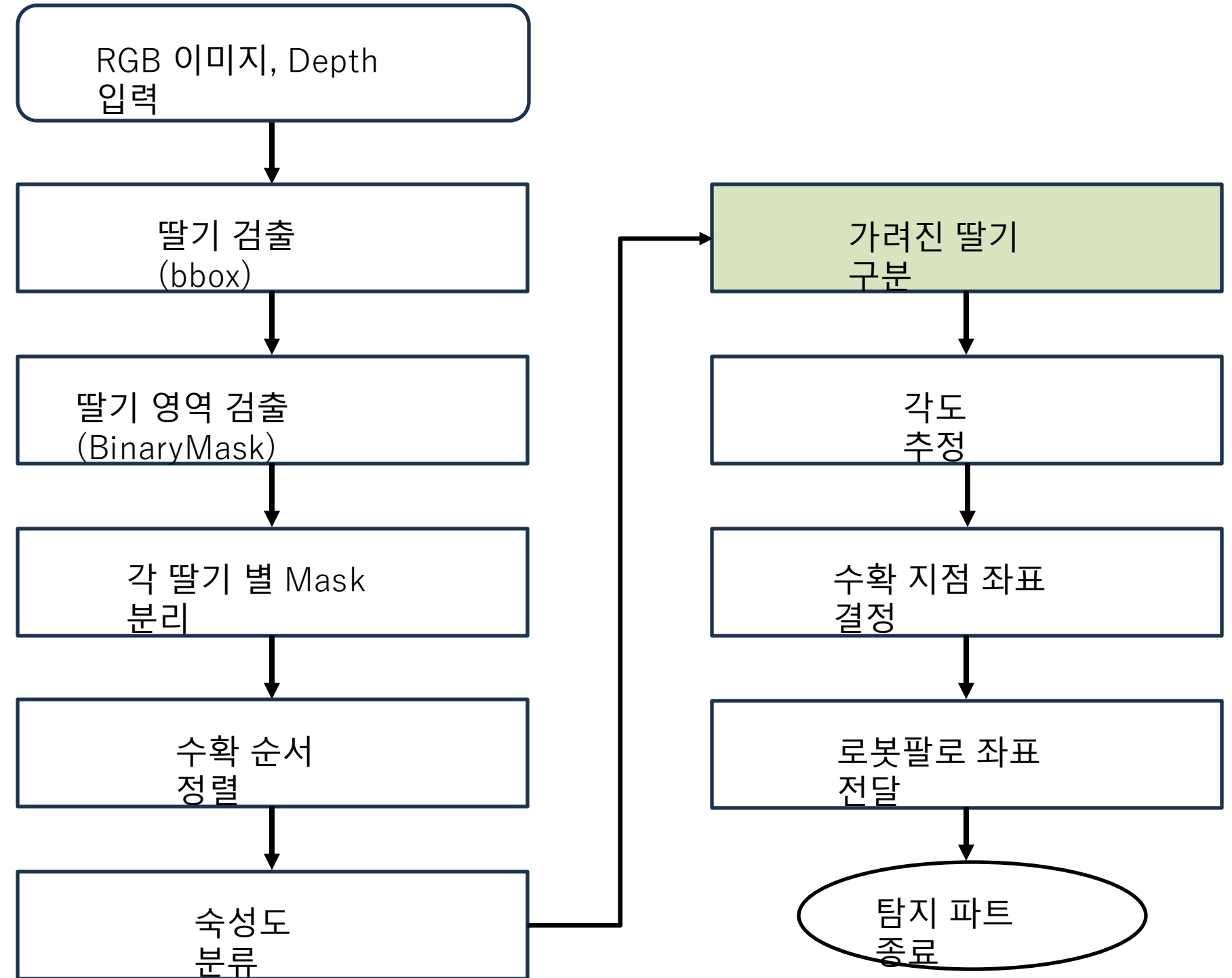




탐지 파트

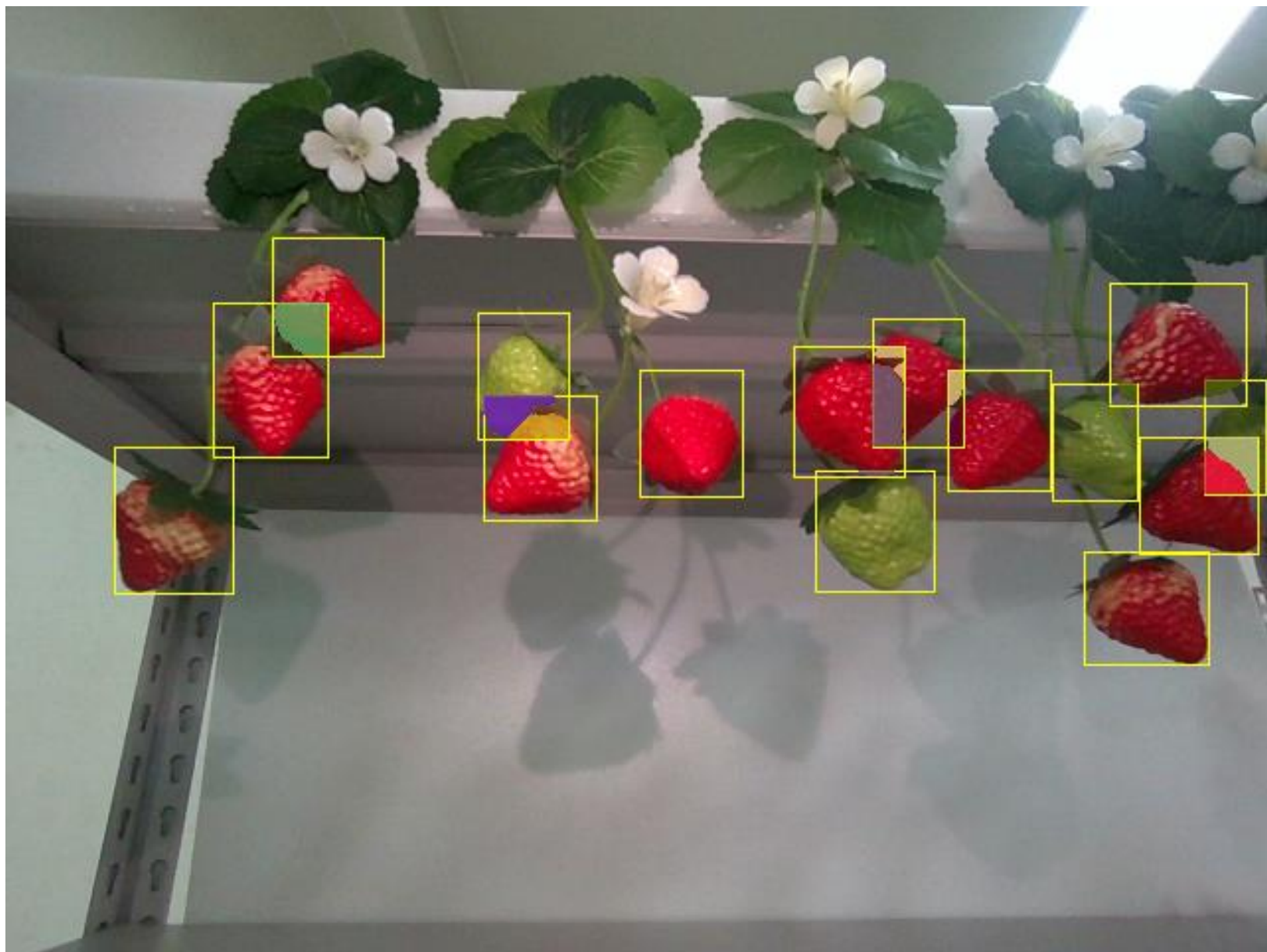
가려진 딸기 구분

- Bbox가 겹쳐져있는 딸기 선택
- 겹친 딸기들의 Depth 값을 비교



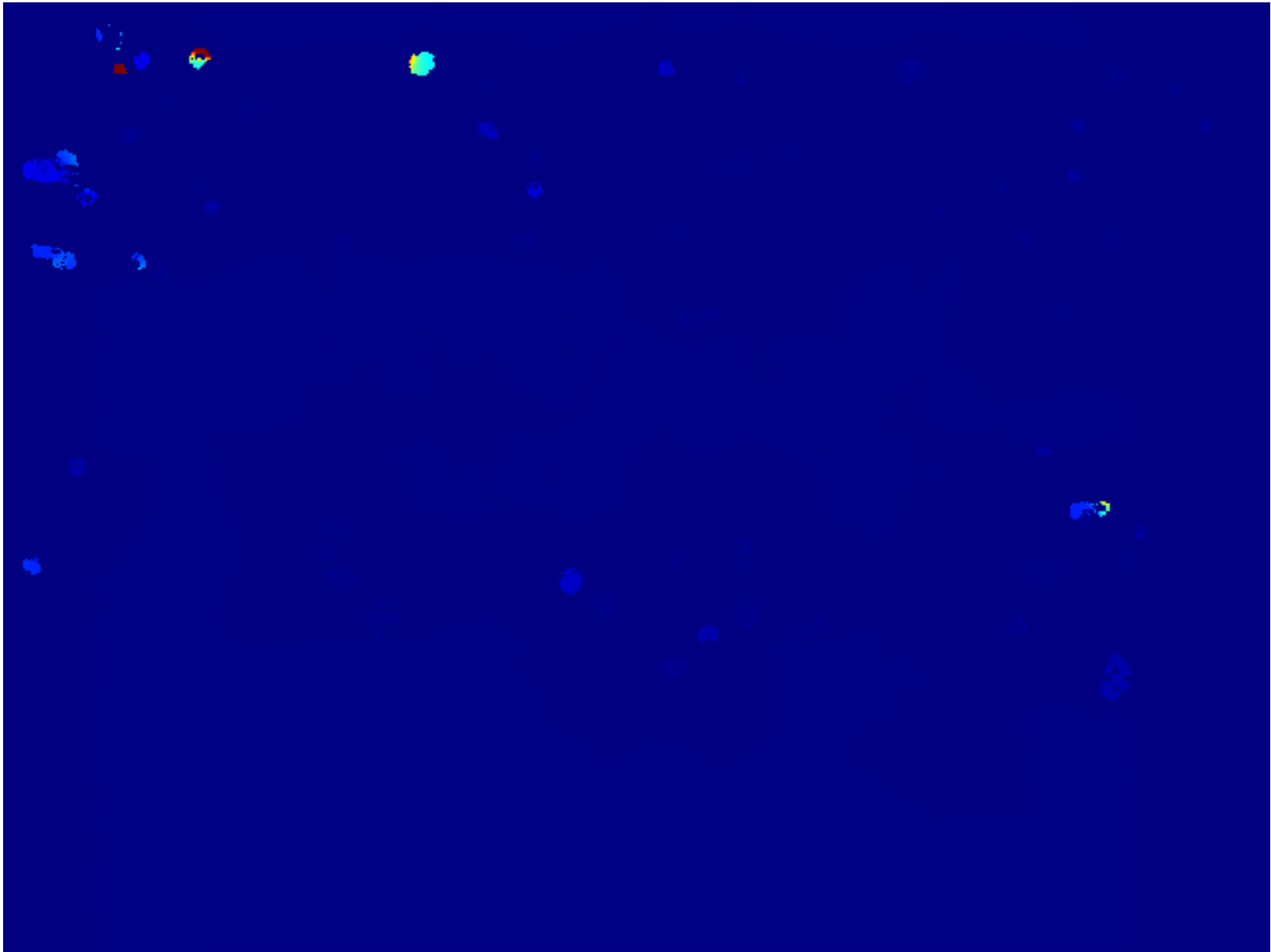


탐지 파트





탐지 파트





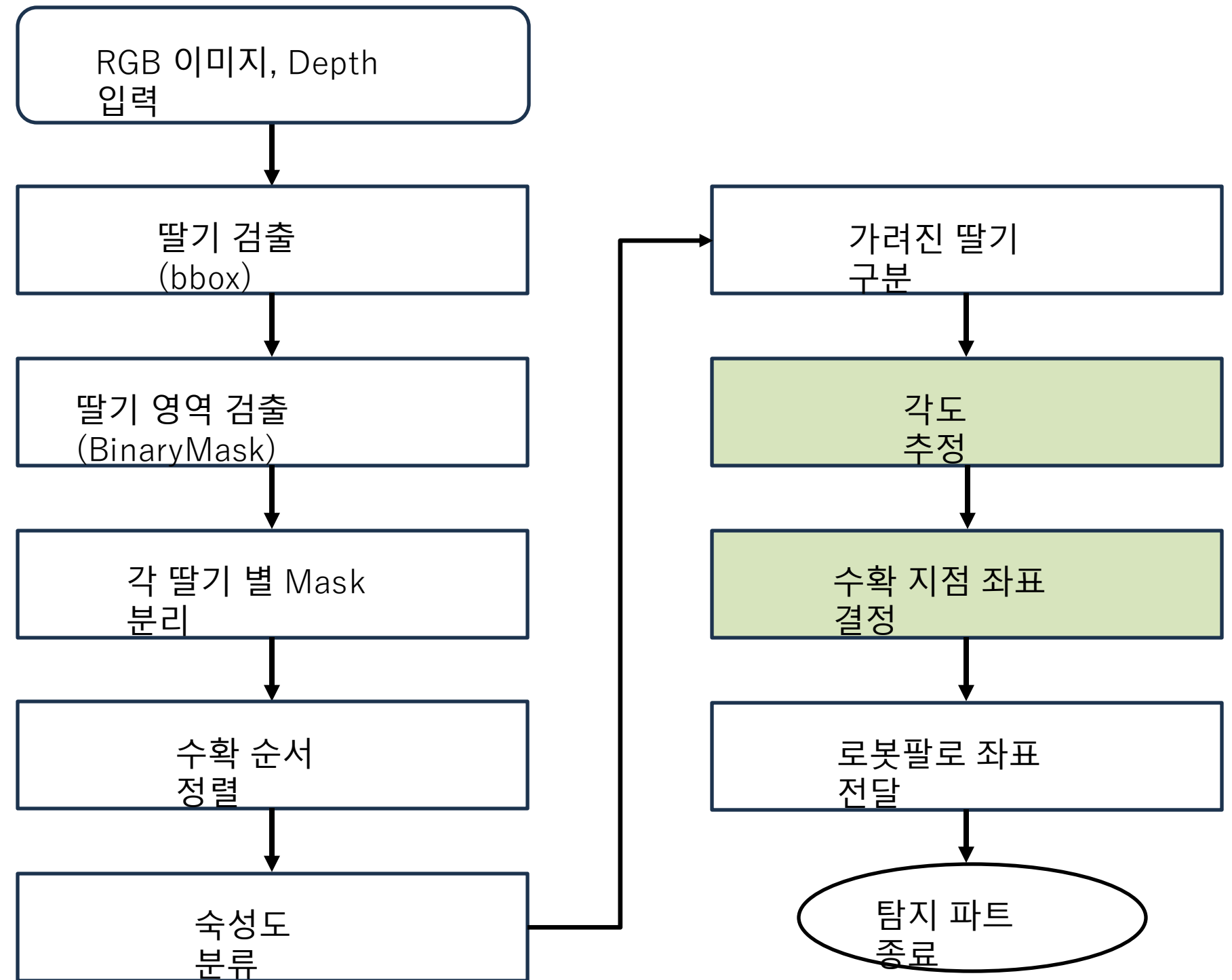
탐지 파트

각도 추정

- 딸기의 수확 지점 결정을 위함.

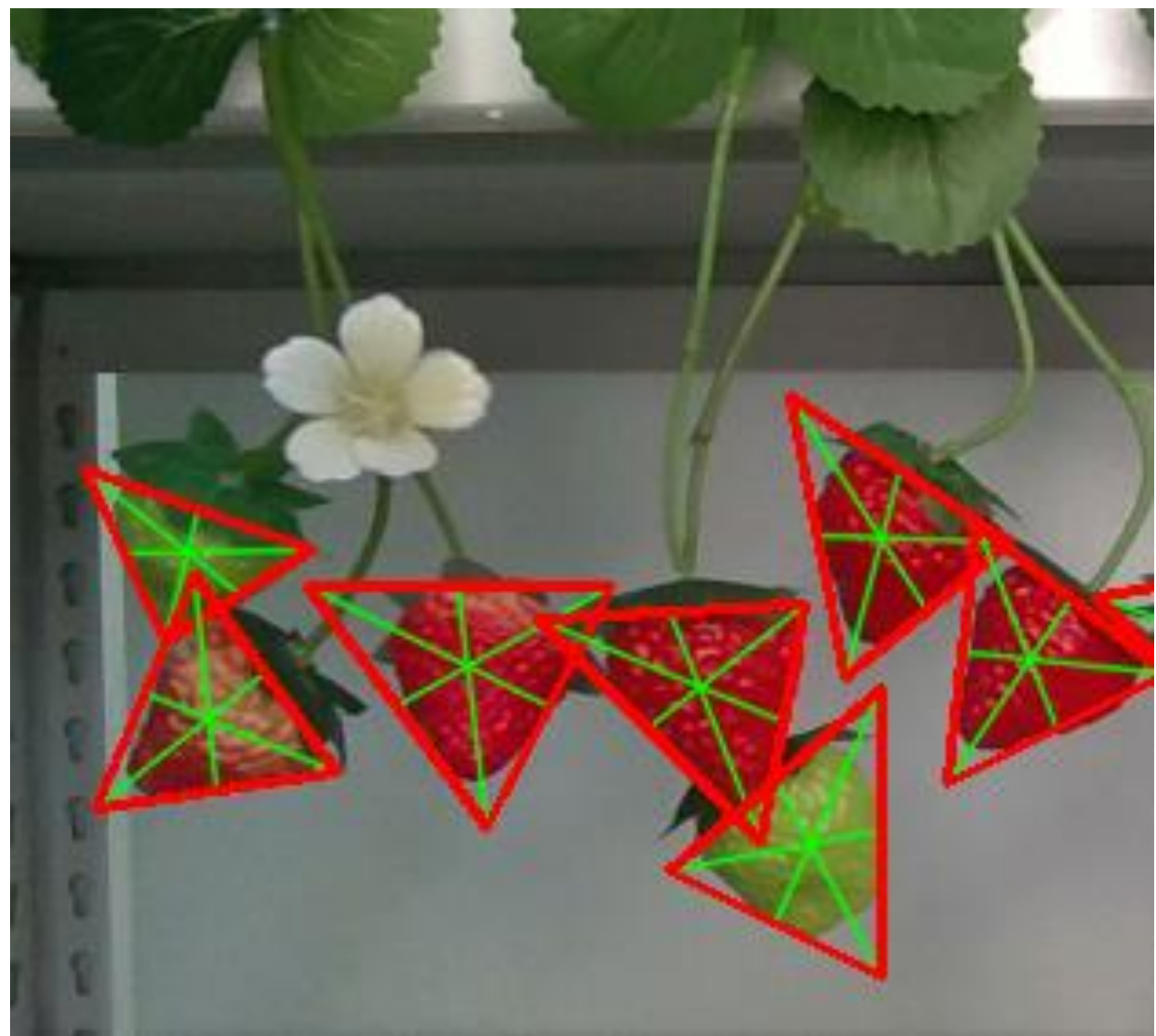
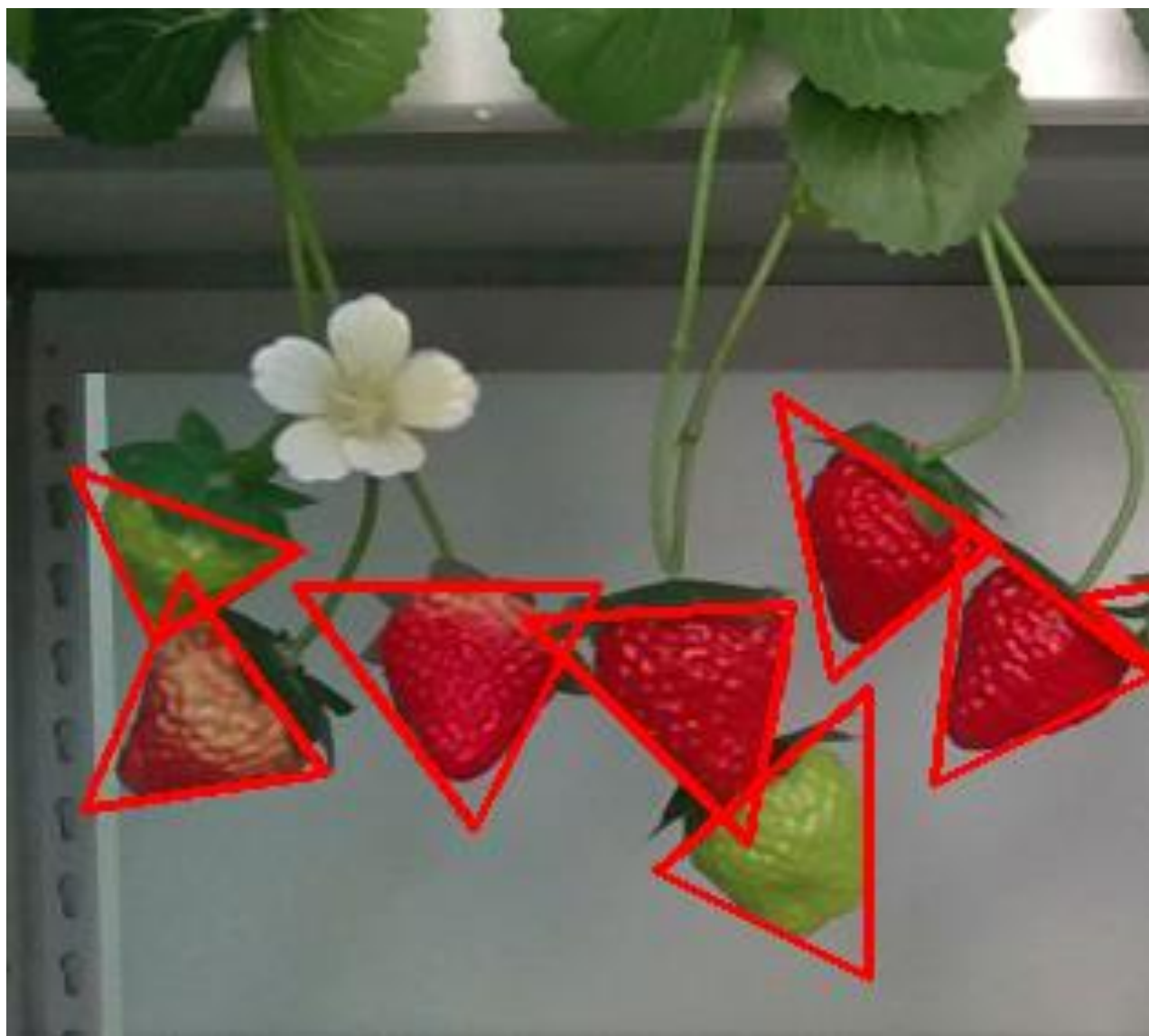
1. 외부 삼각형 피팅
2. 각 꼭짓점에서 중선 긋기
3. 내부 삼각형 찾기
4. 형태를 비교하여 딸기의 가장 아랫부분

추정



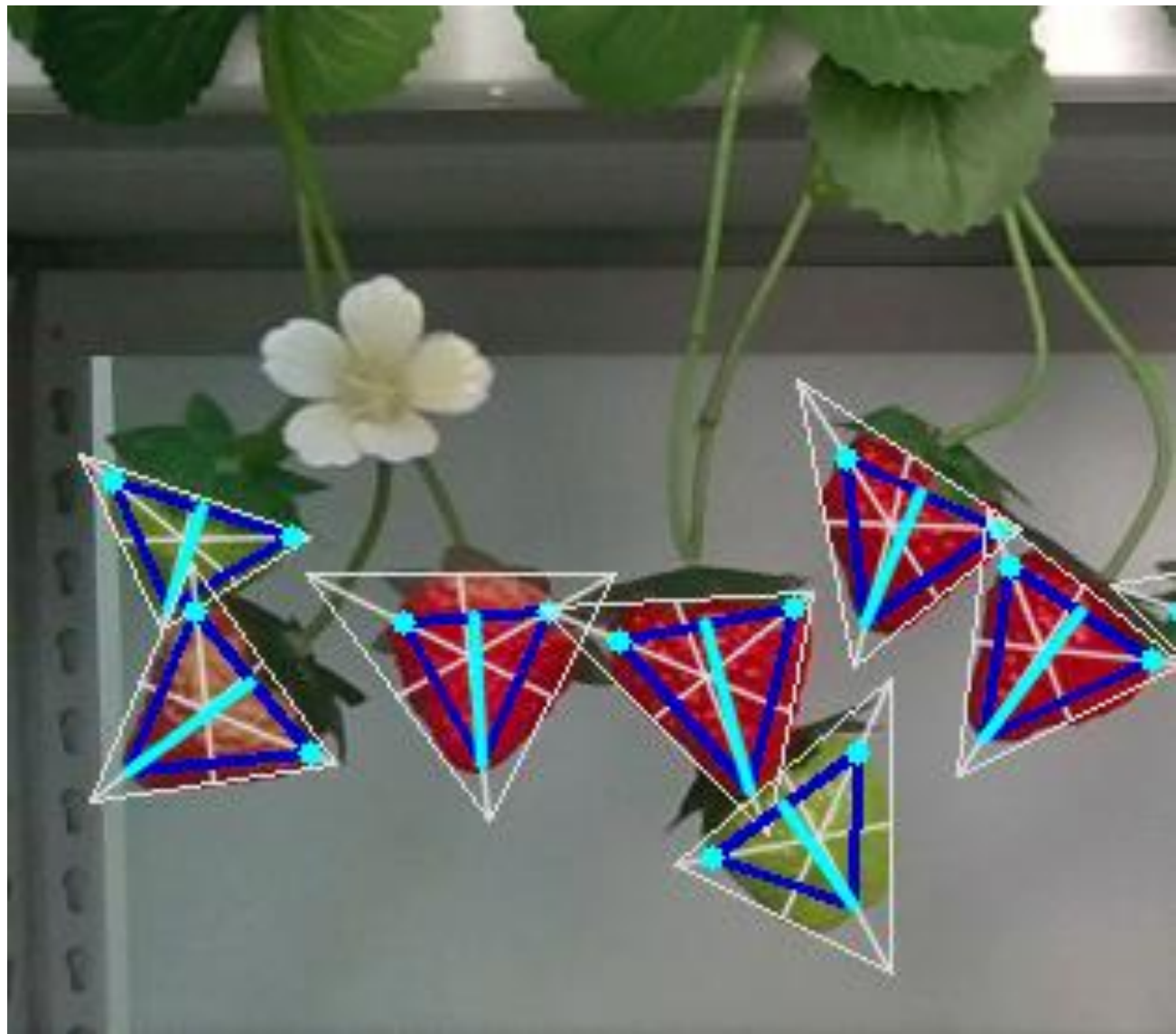


탐지 파트





탐지 파트



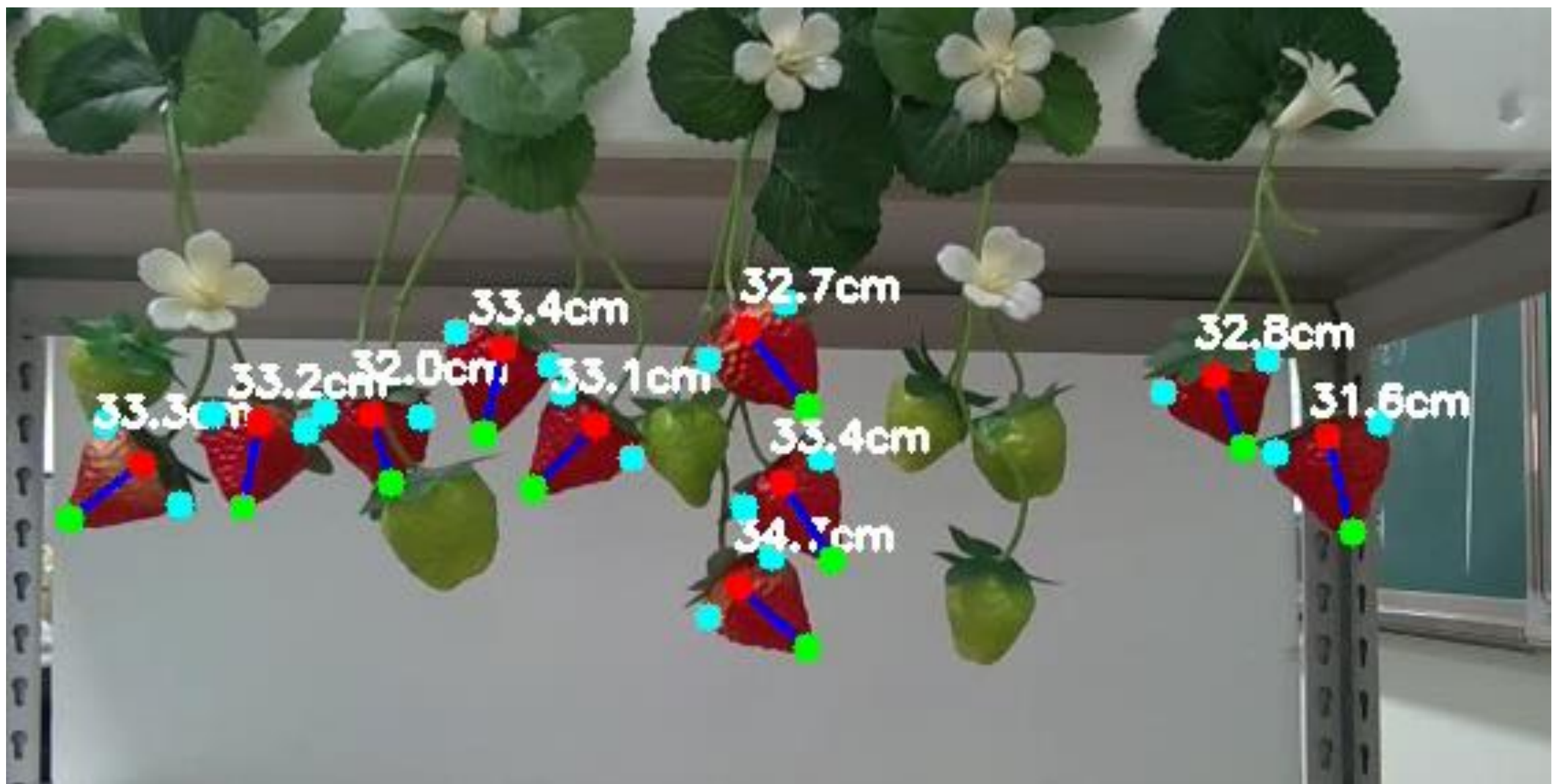


최종 결과





최종 결과



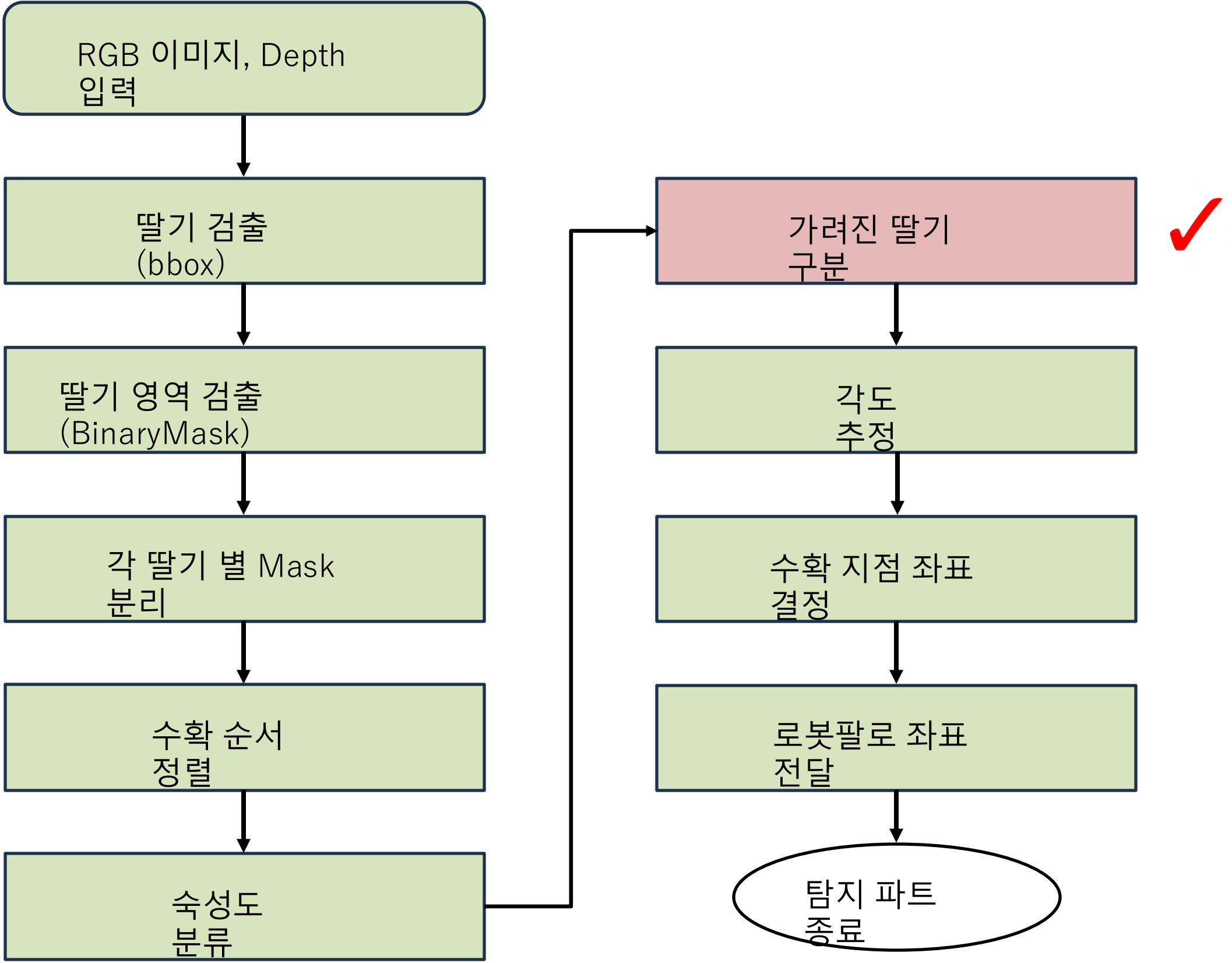
```
{  
  "left": {  
    "x": 0.123,  
    "y": -0.045,  
    "z": 0.287  
  },  
  "right": {  
    "x": 0.130,  
    "y": -0.050,  
    "z": 0.285  
  },  
  "angle": 42.5  
}
```



탐지 파트 향후 계획



탐지 파트





제어 파트



제어 파트

하드웨어

- Neuromeka Indy 7
- OpenMANIPULATOR-X

제어 환경

- Python + IndyDCP2 라이브러리

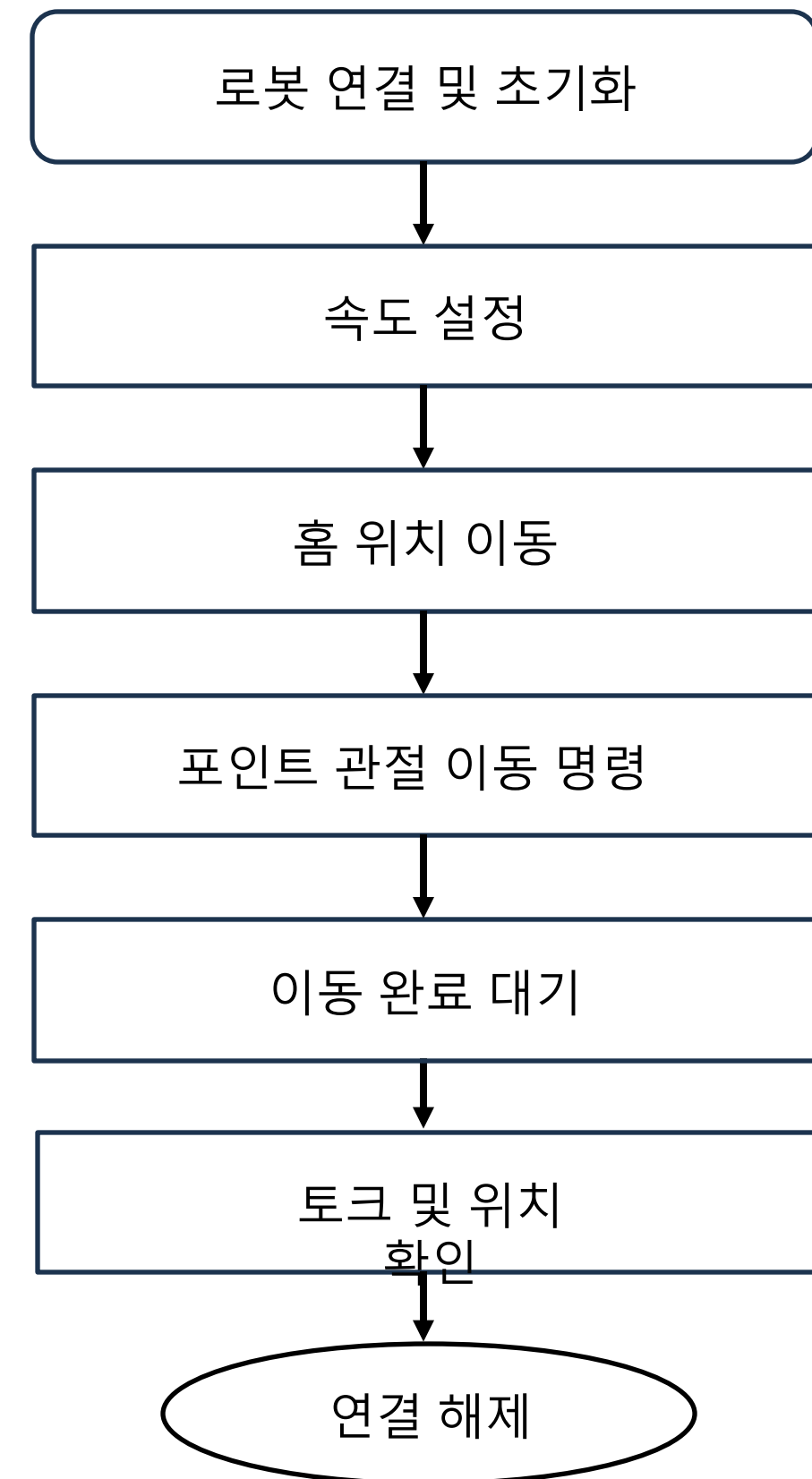




제어 파트

로봇팔 내부 제어 흐름

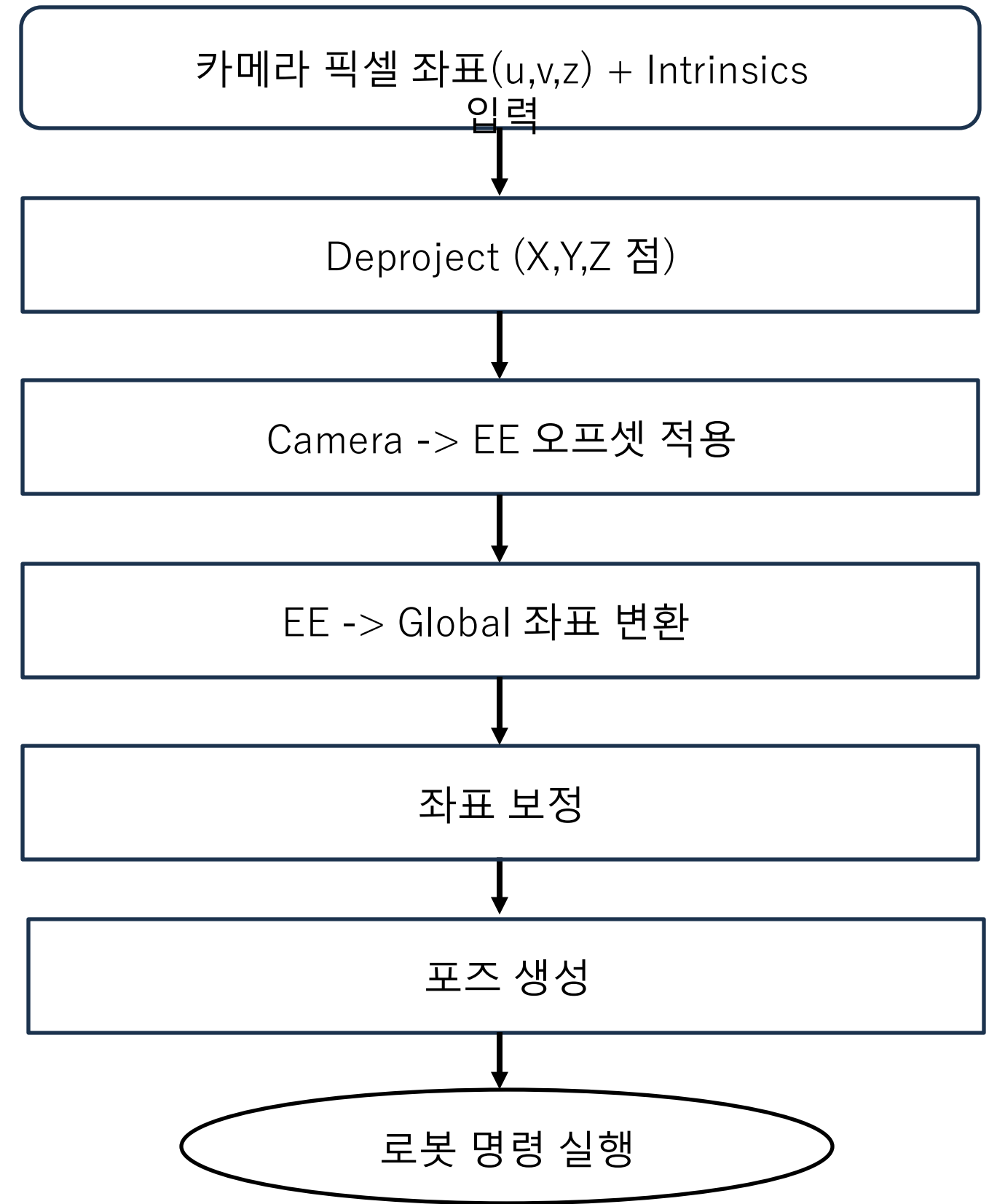
- IndyDCP2 라이브러리를 이용한 기본 제어 흐름





제어 파트

좌표계/행렬 변환 순서도

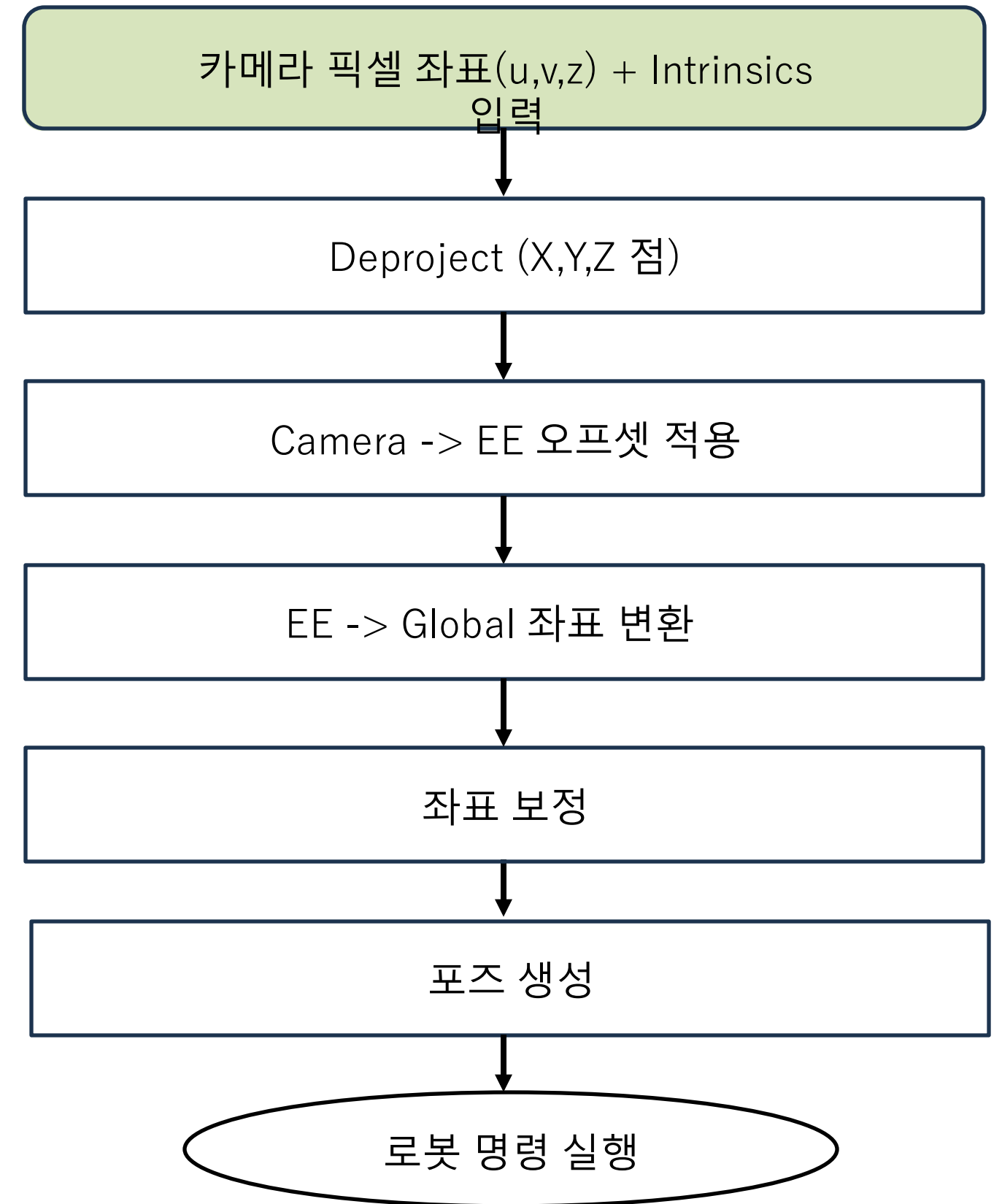




제어 파트

카메라 픽셀 좌표(u,v,) + Z + Intrinsics 입력

- 카메라 픽셀 좌표(u,z) + 깊이 Z
- 카메라 내부 파라미터 Intrinsics 입력
 - f_x : 391.56951904296875
 - f_y : 391.56951904296875
 - c_x : 320.3621826171875
 - c_y : 242.872802734375



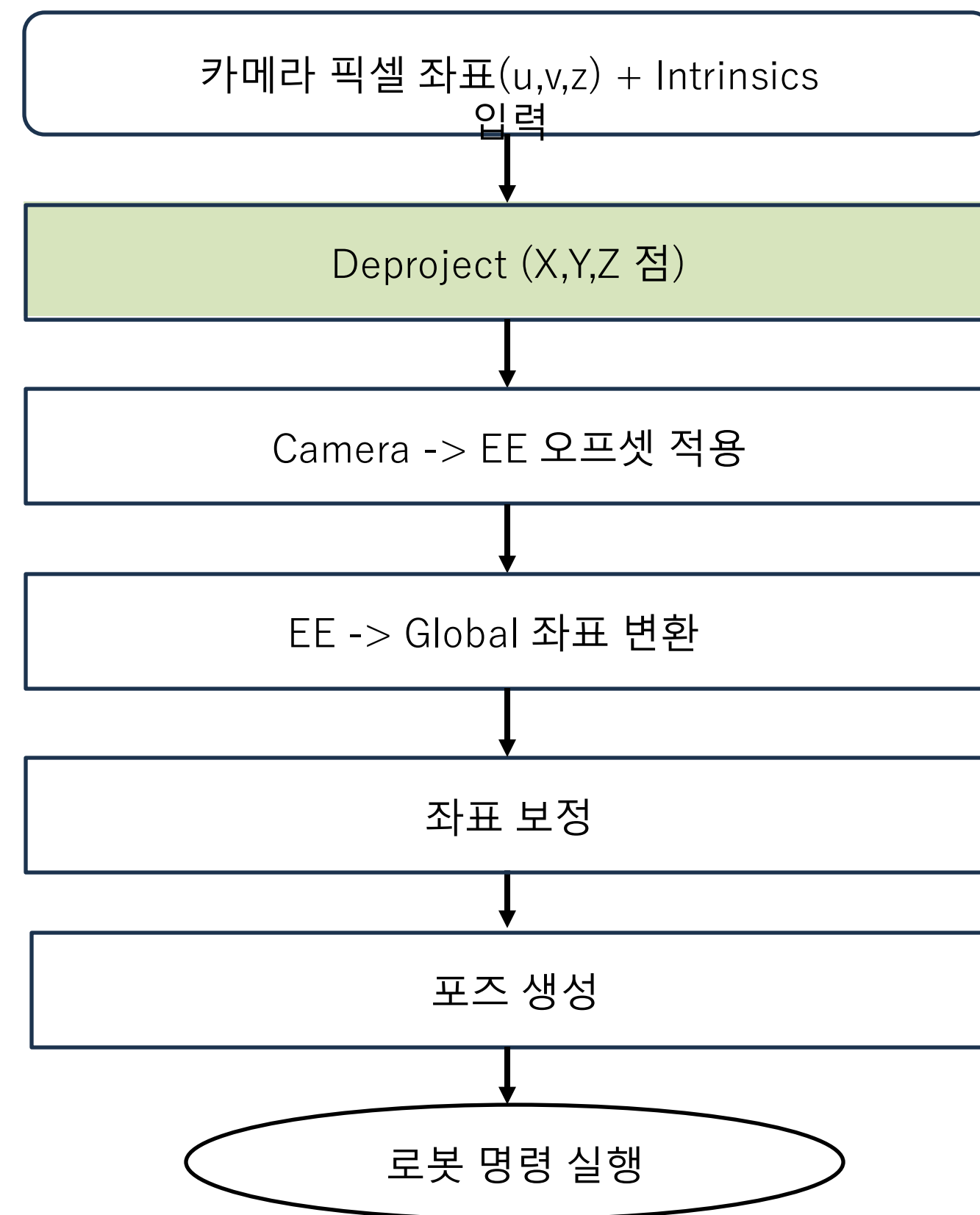


제어 파트

Deproject (카메라 3D 점 얻기)

- 역투영 Deproject

$$\begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix} = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{u-c_x}{f_x} \\ \frac{v-c_y}{f_y} \\ 1 \end{bmatrix}$$
$$\Rightarrow p_C = Z \cdot \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{(u-c_x)Z}{f_x} \\ \frac{(v-c_y)Z}{f_y} \\ Z \end{bmatrix}$$

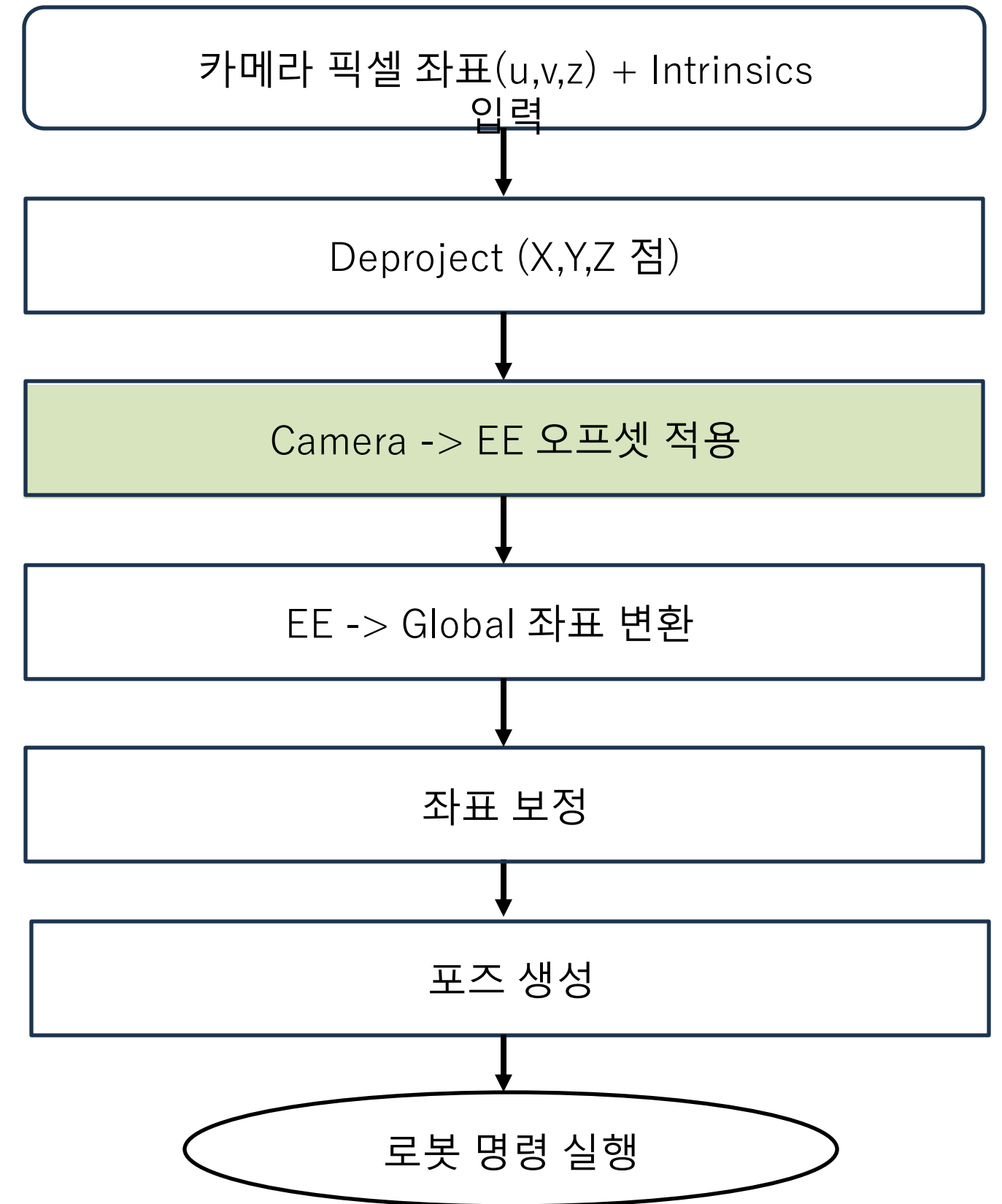




제어 파트

Camera -> EndEffector 오프셋

- 자로 직접 측정한 번역(0.10, 0.00, 0.127m) 적용



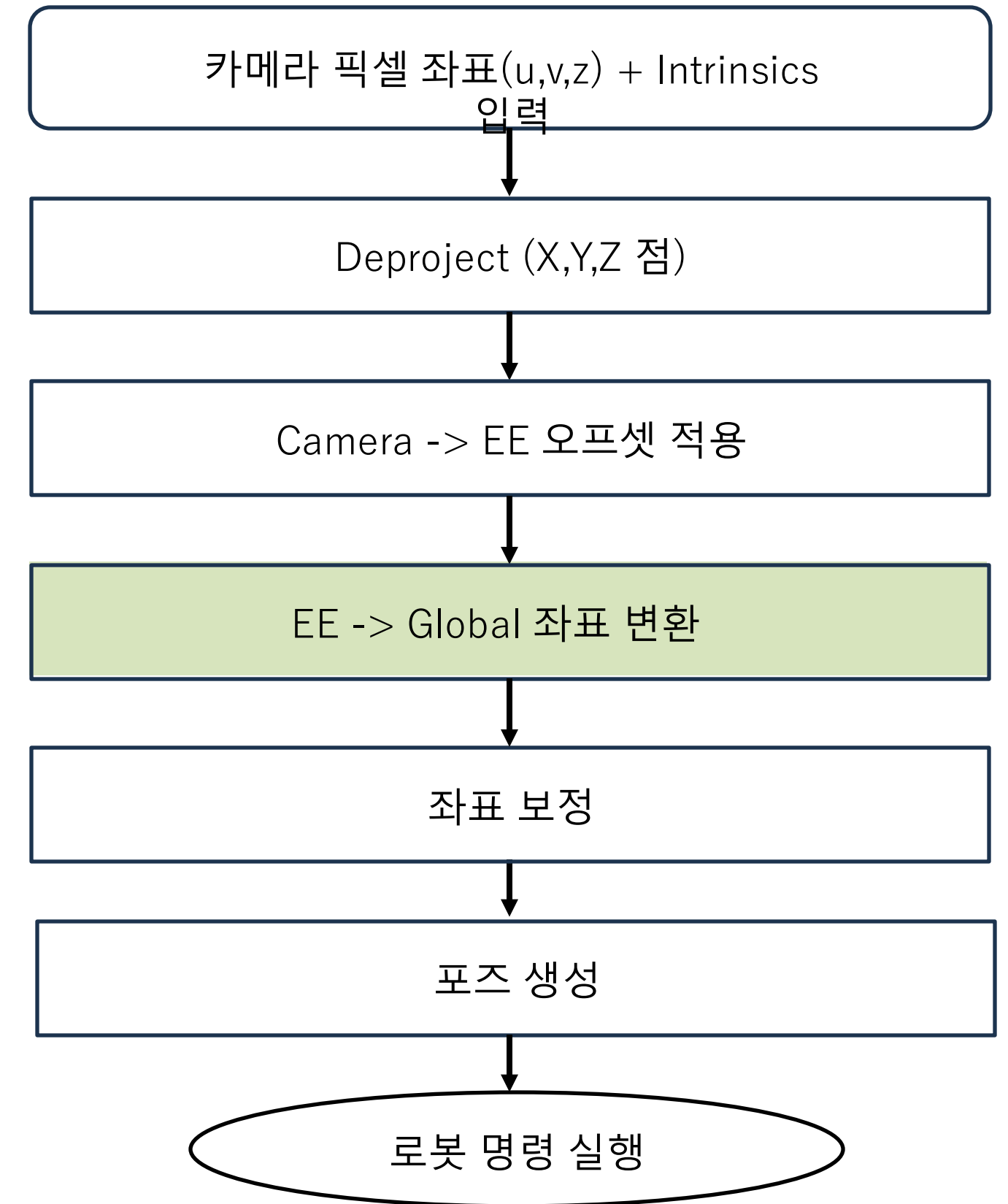


제어 파트

EndEffector -> Global 변환

- 로봇에서 읽은 현재 태스크 포즈 $[x, y, z, r_x, r_y, r_z]$ 를
행렬로 변환

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}, R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix},$$
$$R_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}, R_{B \leftarrow E} = R_z(\gamma) R_y(\beta) R_x(\alpha)$$
$$T_{B \leftarrow E} = \begin{bmatrix} R_{B \leftarrow E} & \begin{bmatrix} x \\ y \\ z \end{bmatrix} \\ 0 & 1 \end{bmatrix}.$$





제어 파트

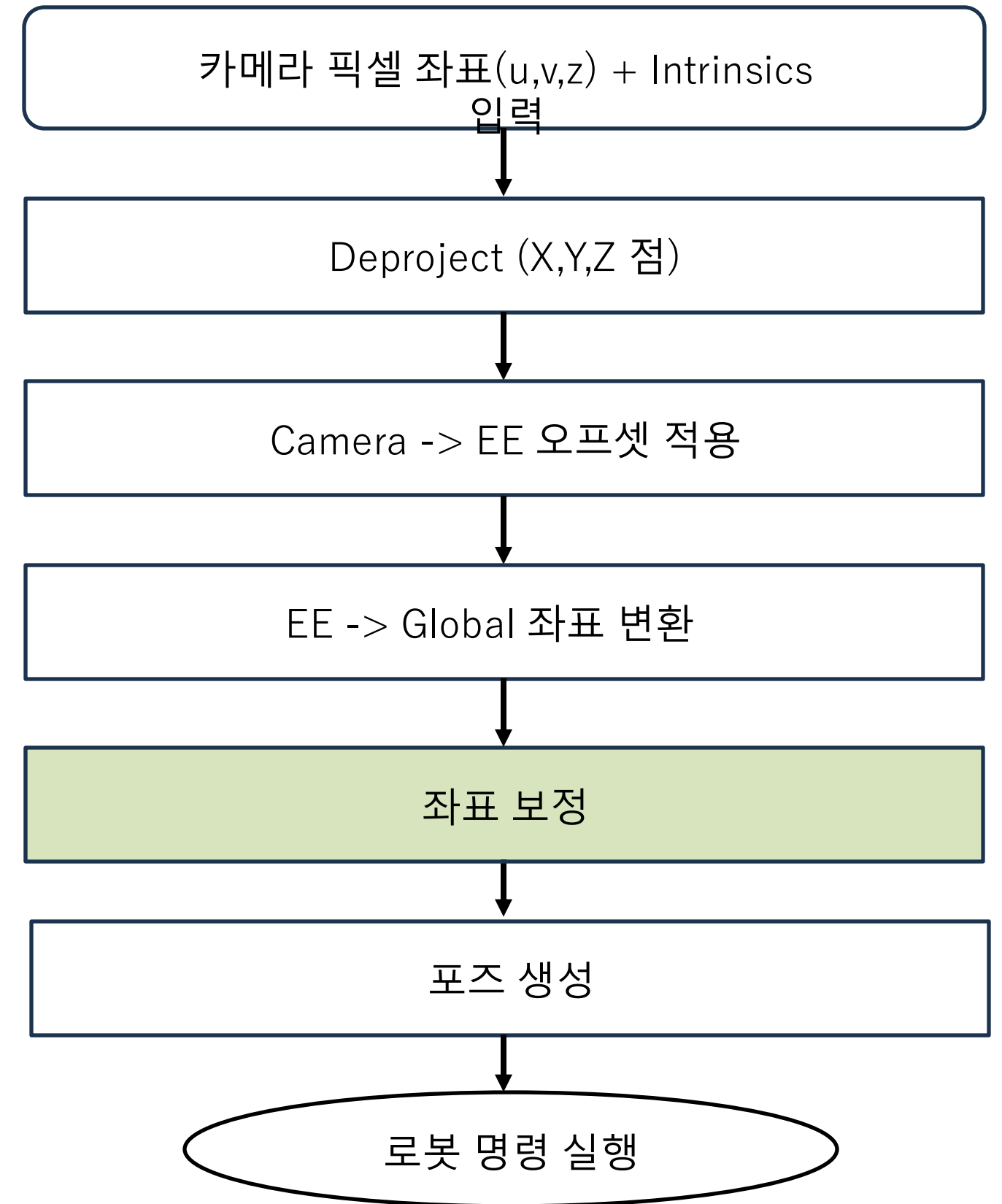
좌표 보정(Backoff, Lift 등)

- Backoff

$$p_B^{back} = p_B^{raw} - d \hat{z}_E^{(B)} \quad (d > 0, \text{ m})$$

- Lift

$$p_B^{final} = p_B^{back} + \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix} \quad (h > 0, \text{ m})$$





제어 파트

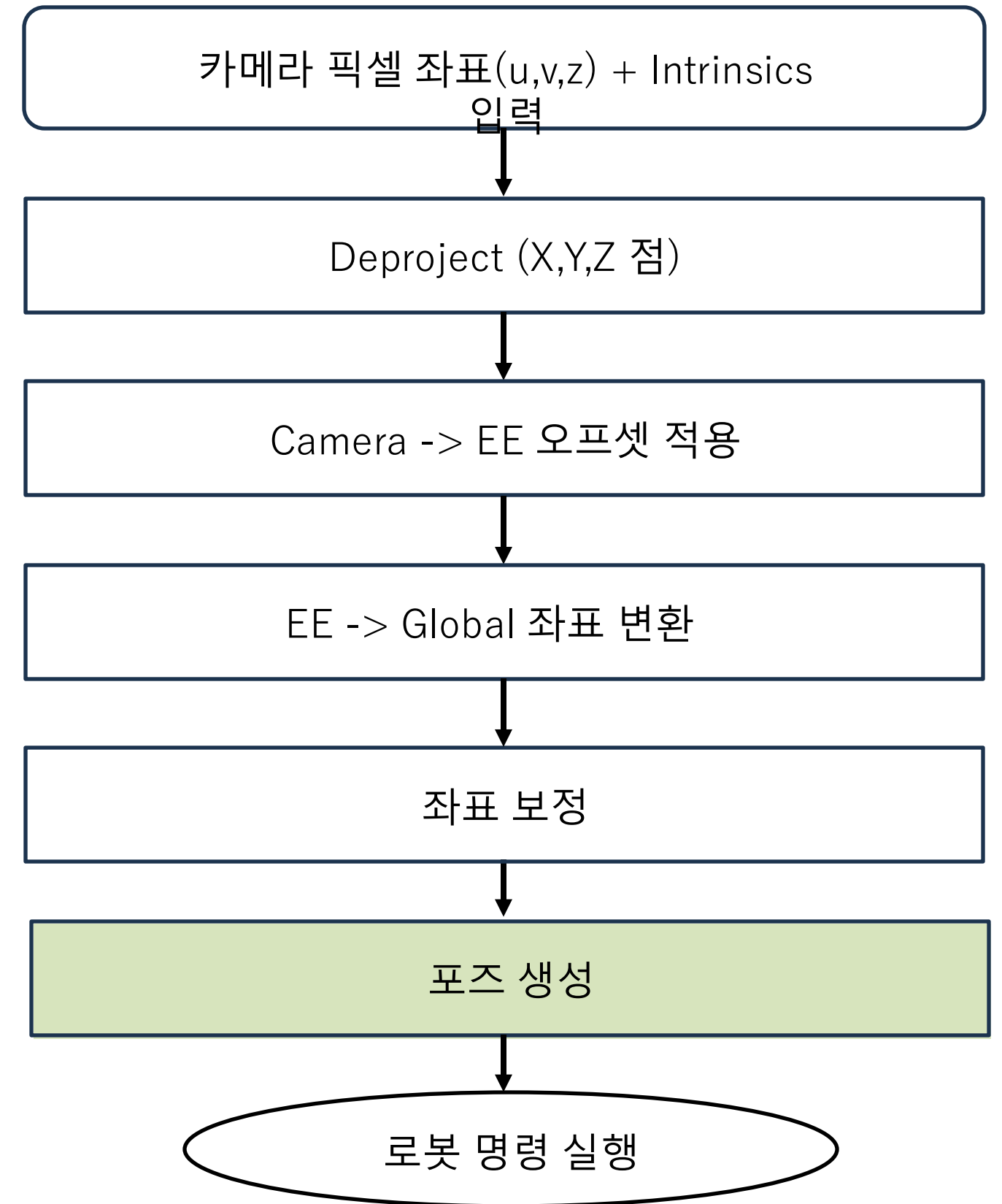
좌표 보정(Backoff, Lift 등)

- 타겟 방향으로 EE z축 정렬

$$\hat{z} = \frac{p_{\mathbf{B}}^{\text{final}} - t_{\mathbf{B} \leftarrow \mathbf{E}}}{\|\cdot\|}, \quad \hat{x} = \frac{\hat{t} \times \hat{z}}{\|\cdot\|} \quad (\hat{t} = [0, 0, 1]^T \text{ 또는 대체벡터}), \quad \hat{y} = \hat{z} \times \hat{x}, \quad R_{\text{target}} = [\hat{x} \ \hat{y} \ \hat{z}].$$

- 최종 변환 & 접근 포즈

$$T_{\text{target}} = \begin{bmatrix} R_{\text{target}} & p_{\mathbf{B}}^{\text{final}} \\ 0 & 1 \end{bmatrix}, \quad T_{\text{approach}} : z \leftarrow z + \Delta z \quad (\text{Base} + Z, \Delta z \approx 0.05 \text{ m})$$





제어 파트

로봇 명령 실행(IndyDCP2 라이브러리)

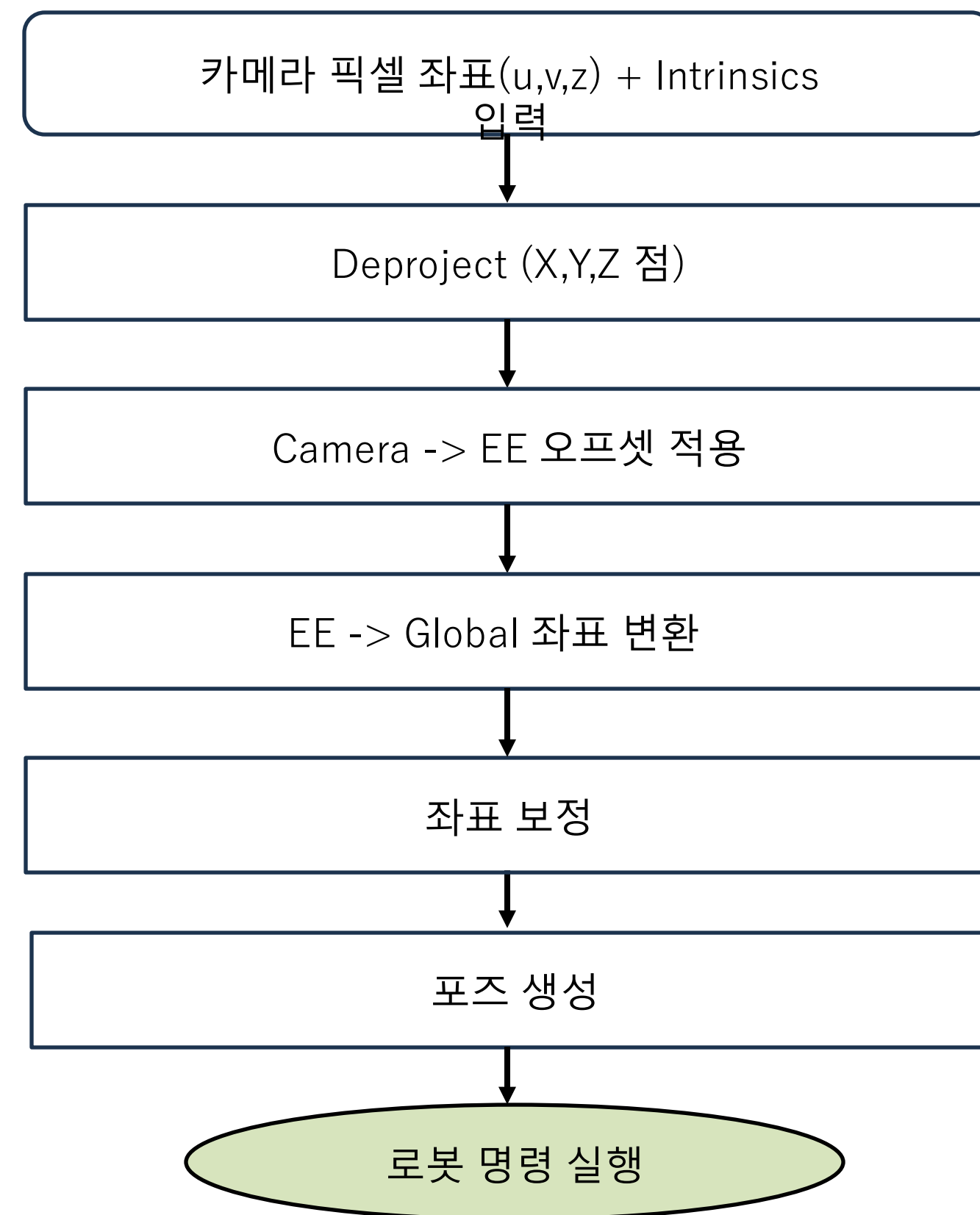
- 회전행렬 $\rightarrow$ RPY(ZYX, deg)

$$\begin{aligned}\beta &= \arcsin(-R_{3,1}), \\ \alpha &= \begin{cases} \text{atan2}(R_{3,2}, R_{3,3}), & |\cos \beta| > \varepsilon \\ \text{atan2}(-R_{2,3}, R_{2,2}), & \text{그 외} \end{cases} \\ \gamma &= \begin{cases} \text{atan2}(R_{2,1}, R_{1,1}), & |\cos \beta| > \varepsilon \\ 0, & \text{그 외} \end{cases}\end{aligned}$$

- approach_pose & target_pose

approach_pose = $[x, y, z, rx, ry, rz]_{\text{deg}}$ from T_{approach}

target_pose = $[x, y, z, rx, ry, rz]_{\text{deg}}$ from T_{target}





제어 파트





제어 파트 향후 계획



제어 파트 향후 계획

실시간 보정 알고리즘

- 좌표 변환 공식 고도화 및 실시간 보정 알고리즘 개발 계획

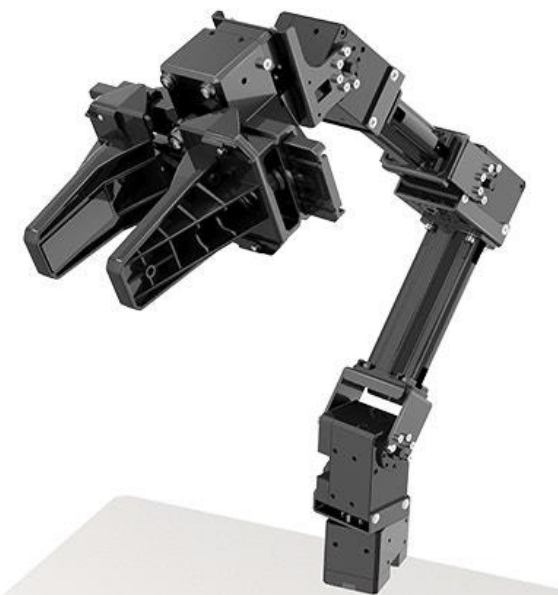


제어 파트 향후 계획

로봇손(OpenMANIPULATOR-X)

- Jetson Nano를 메인 컨트롤러로 사용
- INDY7: TCP/IP 기반 제어 (Neuromeka IndyDCP SDK 사용)
- OpenMANIPULATOR-X: U2D2 및 Dynamixel SDK를 통한 USB 시리얼 기반 제어
- 통신: 포트(/dev/ttyUSB0), 1Mbps, Protocol 2.0, TTL 3-핀(JST) 사용
- 흐름도

Vision → 좌표 추정 → INDY7 위치 이동 → OpenMANIPULATOR-X 그리퍼 동작



OpenMANIPULATOR-X



제어 파트 향후 계획

Vision + Arm + Gripper 통합 제어 시스템 구축

- 딸기 수확 전체 과정 자동화 목표
- 딸기 위치 좌표 -> 로봇 기준 좌표로 변환
- 로봇 손 제어 알고리즘: 적절한 힘, 타이밍 조절
- 탐지 -> 이동 -> 수확 -> 복귀 전체 순환 로직 자동화



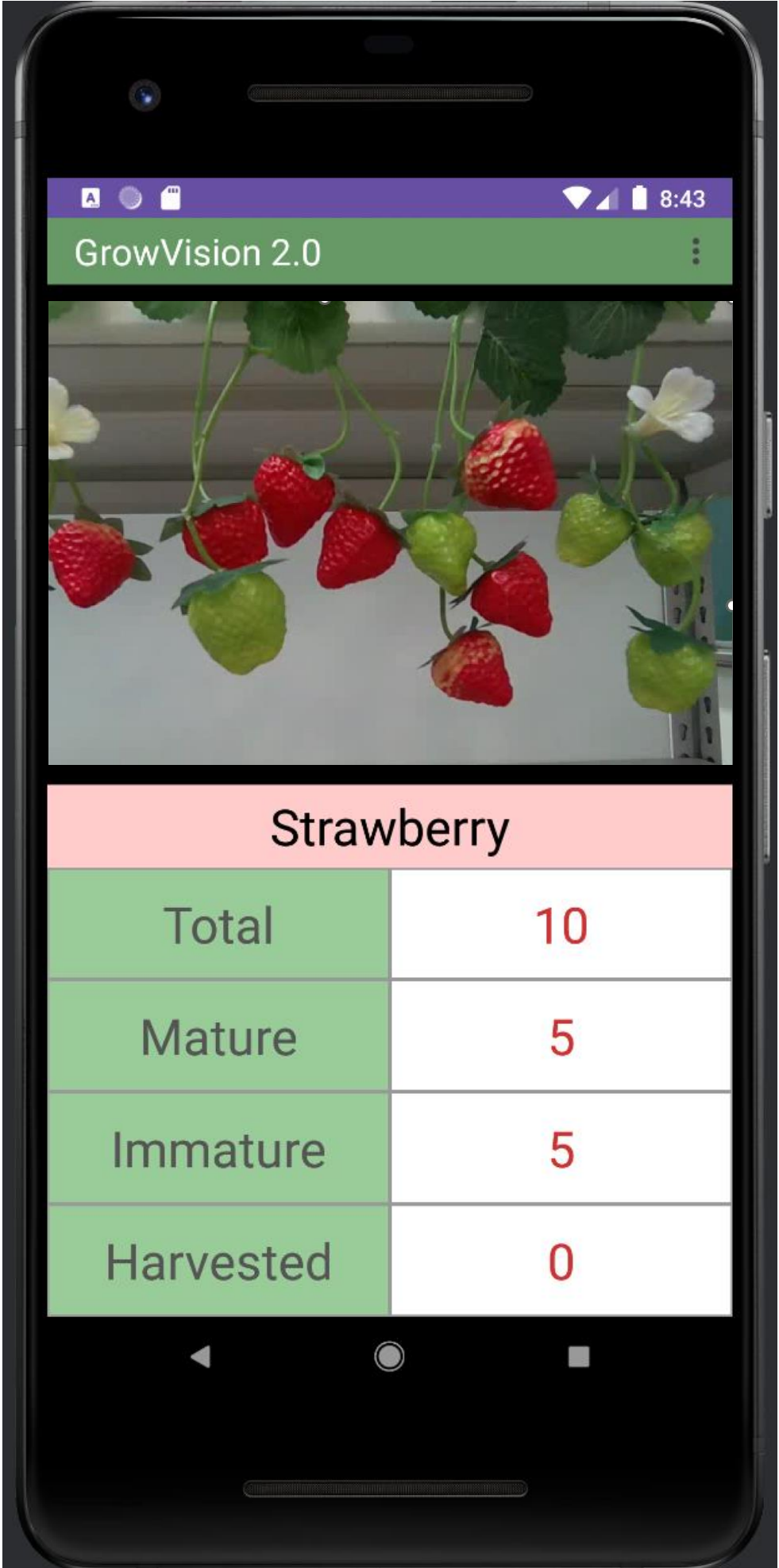
앱 파트



앱 파트

안드로이드 앱

- 카메라 화면 실시간 스트리밍
- 실시간 수확 현황
- 수확 로그

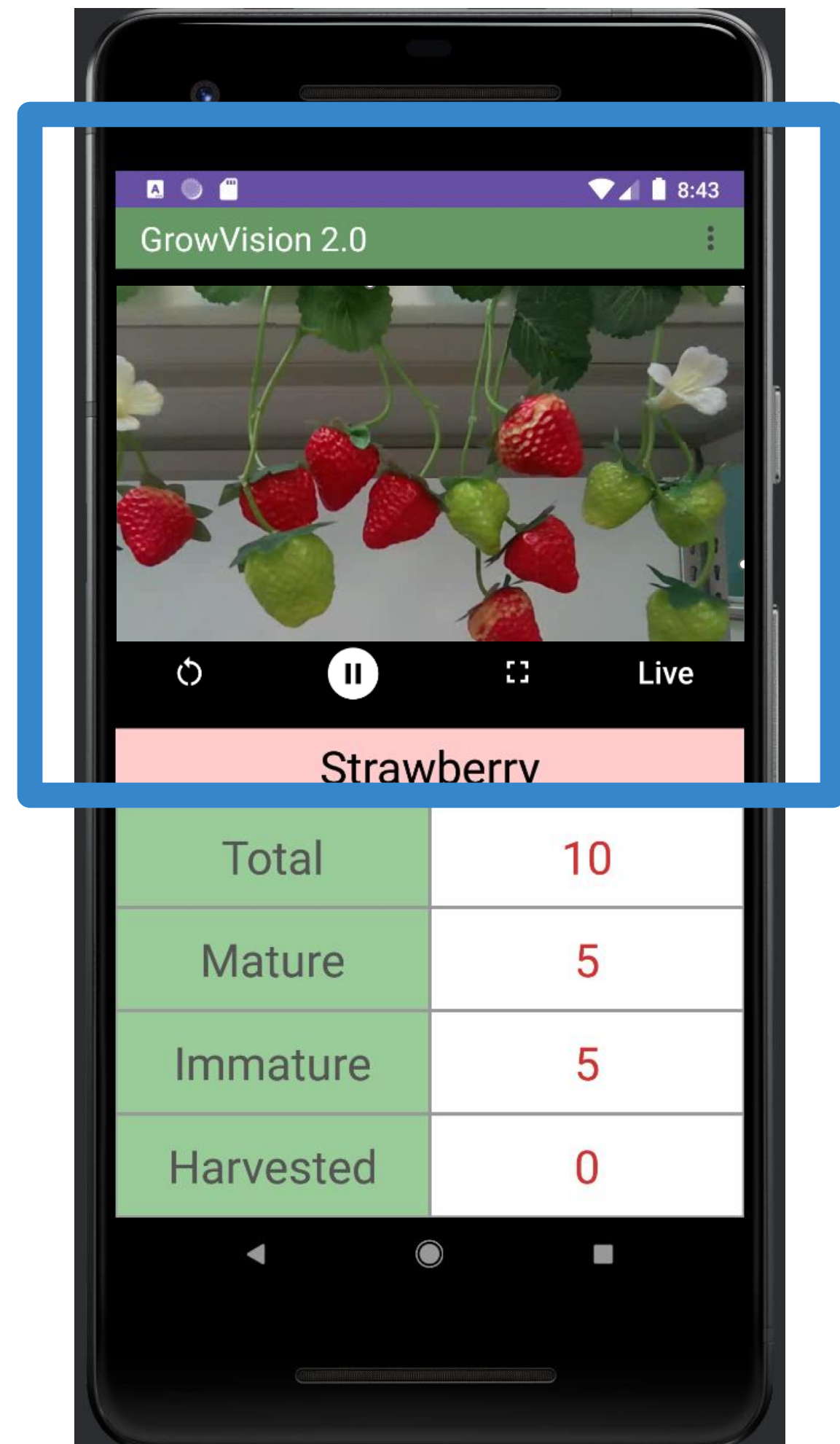




앱 파트

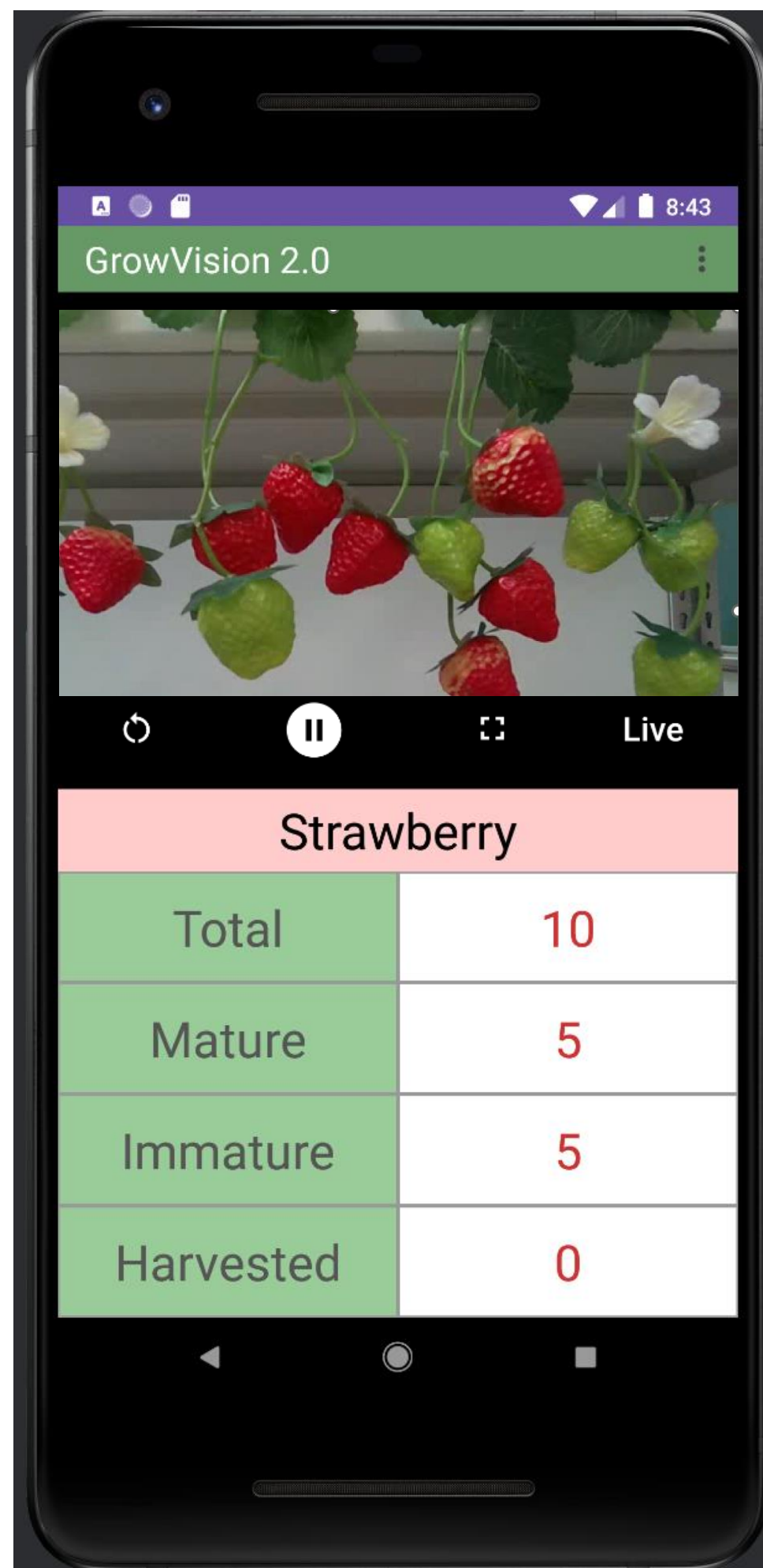
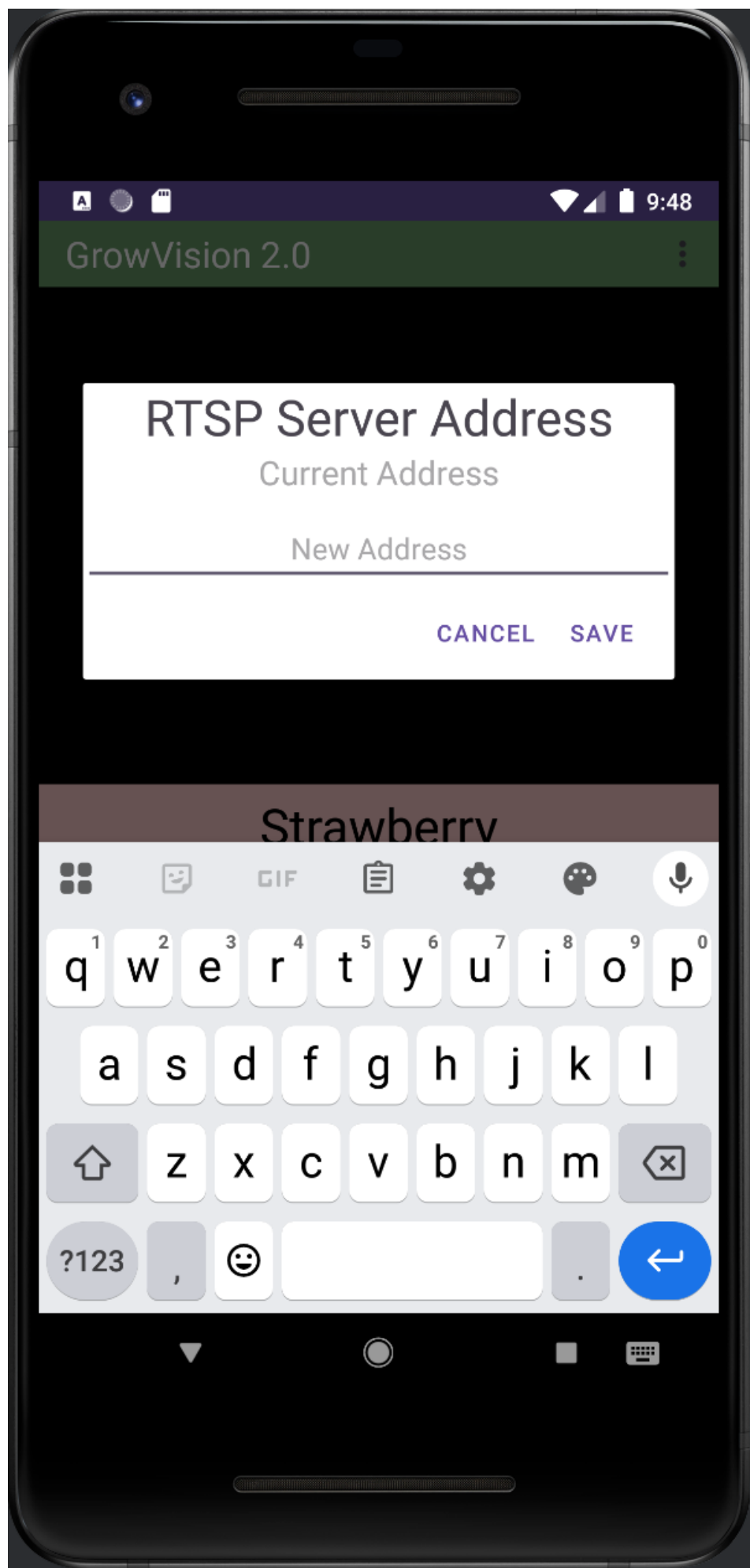
카메라 화면 실시간 스트리밍

- 로봇팔에 장착된 카메라의 화면을 실시간으로 스트리밍
- Jetson에서 RTSP 서버를 통해 송출
- 새로고침, 재생/일시정지, 전체화면, 실시간으로 건너뛰기 등의 기능





앱 파트

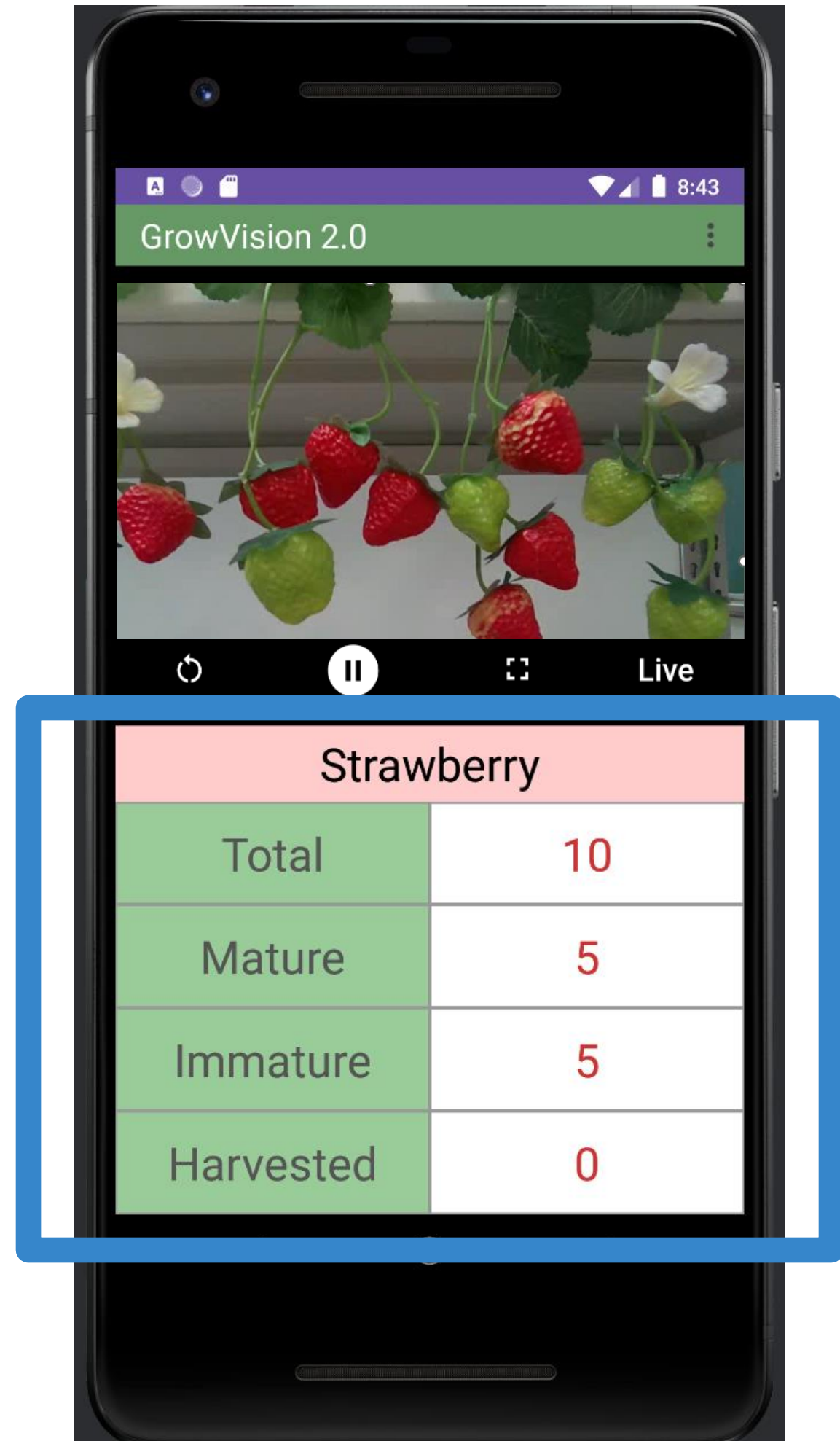




앱 파트

실시간 수확 현황

- 현재 로봇이 감지한 딸기(전체, 숙성, 미숙성) 수량 표시
- 로봇이 딸기를 수확할 때마다 수확한 개수 증가
- Firebase 상에서 데이터 저장

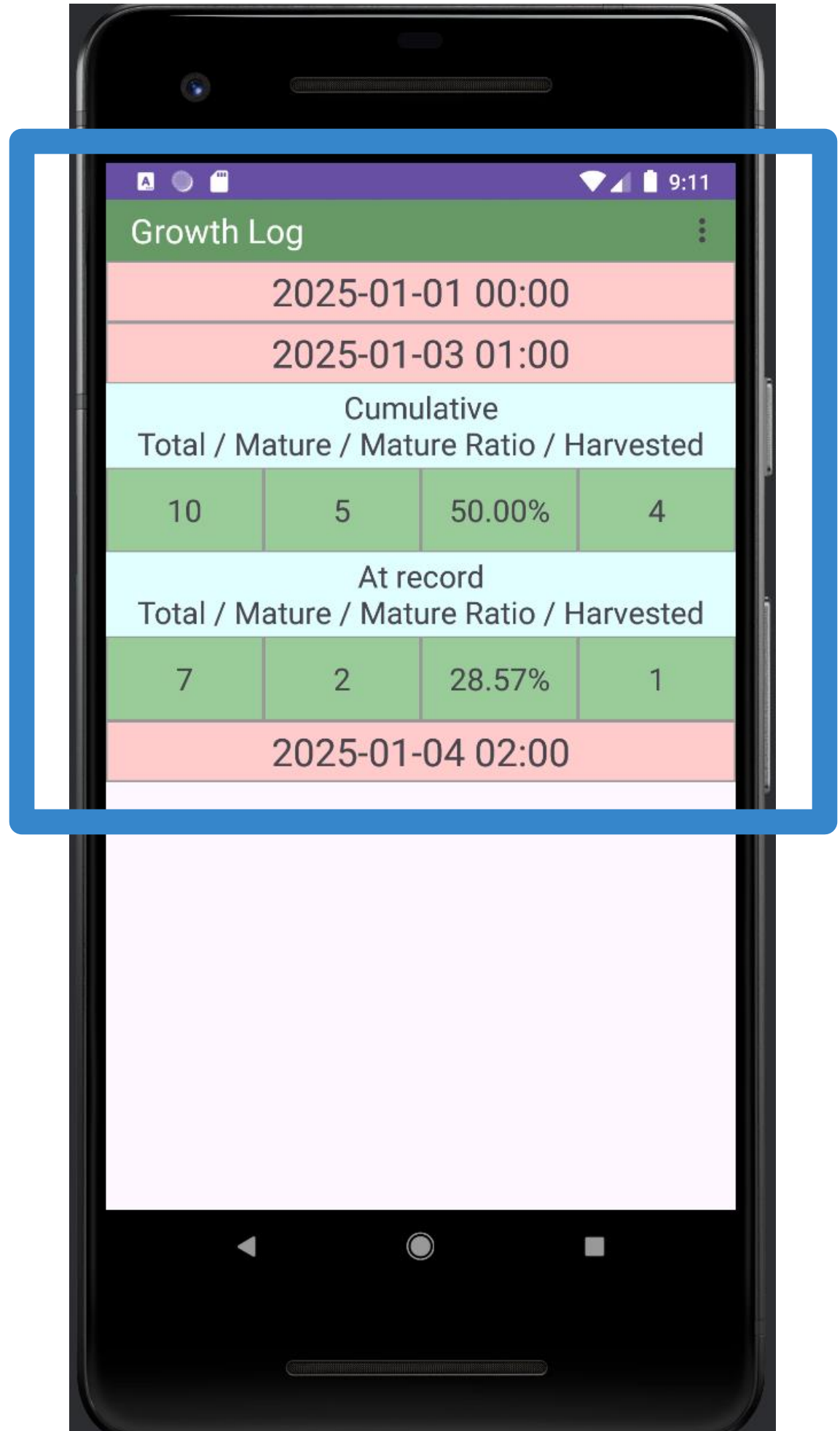




앱 파트

수확 로그

- 로봇 작동 일시를 기준으로 구분하여
누적/로깅시점 별 수확 로그 저장
- 전체, 숙성, 수확한 딸기 수량 및 숙성된 딸기 비율
표시
- 각 항목은 접기/펼치기로 표시
- Firebase 상에서 데이터 저장

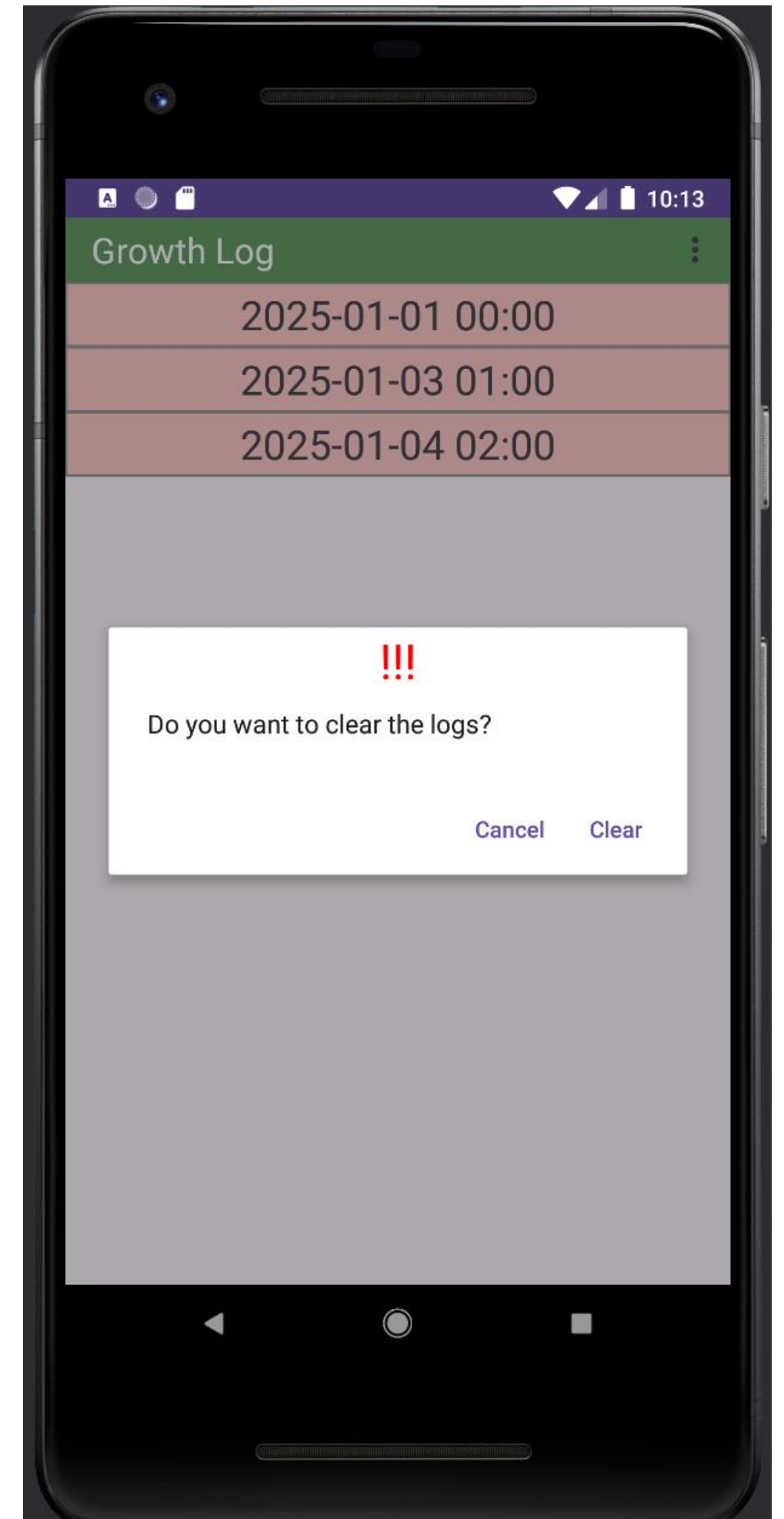




앱 파트

수확 로그

- 로봇 작동 일시를 기준으로 구분하여
누적/로깅시점 별 수확 로그 저장
- 전체, 숙성, 수확한 딸기 개수 및 숙성된 비율 표시
- 각 항목은 접기/펼치기로 표시
- Firebase 상에서 데이터 저장





앱 파트 향후 계획



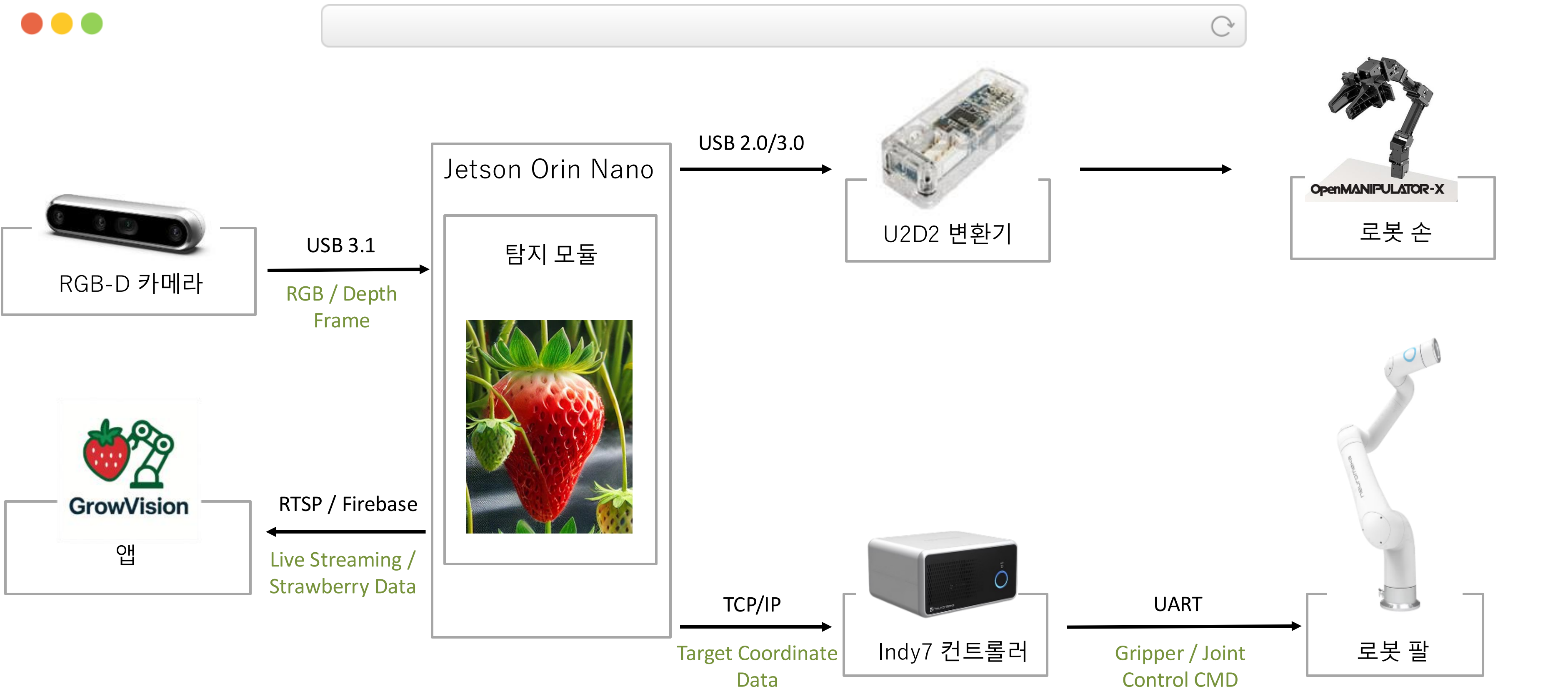
앱 파트 향후 계획

탐지 + 제어 + 앱 파트 통합 시스템 구축

- 탐지 + 제어 시스템과 서버 간 데이터 통신 시스템 구축 목표
- Jetson의 RTSP 서버를 통한 실시간 스트리밍
- 탐지 및 수확 신호 수신 이후 Firebase로 데이터 전송
- Jetson + 서버 + 앱 간 실시간 연계 시스템 확립



최종 시스템 구성도





감사합니다.